ADVANCED SOFTWARE ENGINEERING

CS603 | Dr. Ali Fahim Ni'ma | Master Studio

Multiple-Choice Bank - Chapters 1 to 8

CONTENTS

Ch	Topic	Items	Pages	Starts
Ch 1	**Introduction**	1.1 - 1.20	8	2
Ch 2	**Software Life-Cycle Models**	2.1 - 2.25	8	10
Ch 3	**Software Requirements**	3.1 - 3.35	10	18
Ch 4	**Software Project Planning**	4.1 - 4.35	9	28
Ch 5	**Software Design**	5.1 - 5.25	11	37
Ch 6	**Software Metrics**	6.1 - 6.35	17	48
Ch 7	**Software Reliability**	7.1 - 7.60	38	65
Ch 8	**Software Testing**	8.1 - 8.50	25	103

TOTAL

126 pages - 8 chapters - 8 MCQ chapter packs

Compiled 2026-11-11 - pages copied verbatim from the source chapter packs

ASE CHAPTER 1

T1 · Multiple Choice Questions

Official MCQs 1.1–1.20 with sourced answers · plus an authored set in the same pattern

Rapid self-test — recall prompts (no answers shown)

> Cover the cards above, then answer these from memory. If any item fails, return to its source page.

Define software in its three-component form.
Name the two families of manuals and give two members of each.
List the four software characteristics.
Distinguish product from process in one sentence each.
List the three process metrics and three product metrics.
State the unit of effort, and justify why it is neither "persons" nor "months".
Name the four factors of management dependency **in the required order**.
State the two definitions of software engineering quoted in the text.
Give the four reasons software process improvement fails.
State the percentage of development projects attributed to customized products.

Retrieval set — Part A · Official MCQs (1.1 – 1.20)

1.1 · Software is

(a) Superset of programs · **(b)** Subset of programs · **(c)** Set of programs · **(d)** None of the above

Superset of programs (a) — Answer

Software = Program + Documentation + Operating Procedures. Programs are one component inside software — software is the **superset**, the program the **subset**. Option (b) reads the same relationship backwards.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.2 · Which is NOT the part of operating procedure manuals?

(a) User manuals · **(b)** Operational manuals · **(c)** Documentation manuals · **(d)** Installation manuals

Documentation manuals (c) — Answer

Two separate families.

Operating procedure manuals: user manuals, operational manuals, system overview, beginner's guide tutorial, reference guide, installation guide, system administration guide.

Documentation manuals: analysis/specification, design, implementation, testing, formal specification, context diagram, data flow diagrams, flow charts, entity-relationship diagram, source code listings, cross-reference listing, test data, test results.

Aggarwal, §1.2.2, pp.5–6 (Fig. 1.2, Fig. 1.3).

1.3 · Which is NOT a software characteristic?

(a) Software does not wear out · **(b)** Software is flexible · **(c)** Software is not manufactured · **(d)** Software is always correct

Software is always correct (d) — Answer 📝

Characteristics: software **does not wear out**, software **is not manufactured**, software **is flexible**, and its components are **reusable**. "Always correct" is not a property — it is the exact opposite of what the myths section later warns about.

Aggarwal, §1.2.4, pp.8–10.

ليش (d) هي الفخ ⚡

باقي الخيارات كلها خصائص حقيقية منصوصة، فالسؤال يكافئ اللي **حفظ القائمة**. و«always correct» تسمع مثل مدح — لهذا تعدي على قراءة سريعة. حالات الفشل المذكورة مثل Patriot وY2K توضح عملياً أن البرمجيات مو دائماً صحيحة.

Aggarwal, §1.2.4, pp.8–10; §1.1.1, pp.2–3.

1.4 · Product is

(a) Deliverables · **(b)** User expectations · **(c)** Organization's effort in development · **(d)** None of the above

Deliverables (a) — Answer 📝

"Product: what is delivered to the customer... Basically, it is nothing but a set of deliverables only." Option (c) — the organization's effort in development — is the definition of **process**, not product.

Aggarwal, §1.5.2, p.13.

1.5 · To produce a good quality product, process should be

(a) Complex · **(b)** Efficient · **(c)** Rigorous · **(d)** None of the above

Efficient (b) — Answer 📝

"An efficient process is required to produce good quality products." "If the process is weak, the end product will undoubtedly suffer, but an obsessive over-reliance on process is also dangerous."

Aggarwal, §1.5.2, p.13.

1.6 · Which is NOT a product metric?

(a) Size · **(b)** Reliability · **(c)** Productivity · **(d)** Functionality

Productivity (c) — Answer 📝

"Examples of process metrics include productivity, quality, failure rate, efficiency etc. Examples of product metrics are size, reliability, complexity, functionality etc."

Productivity is a process metric, so it is the one that does not belong. Note that **functionality** is a product metric — which is exactly why it cannot also be the answer here.

Aggarwal, §1.5.4, p.13.

1.7 · Which is NOT a process metric?

(a) Productivity · **(b)** Functionality · **(c)** Quality · **(d)** Efficiency

Functionality (b) — Answer

"Examples of process metrics include productivity, quality, failure rate, efficiency etc." Question 1.6 read in reverse.

Aggarwal, §1.5.4, p.13.

قاعدة واحدة تحل 1.6 و1.7 و1.19 من التمارين

اسأل: **مَن المُقاس؟** إذا القياس يوصف **نشاط البناء** ⇒ مقياس عملية. وإذا يوصف **الشي المبني** ⇒ مقياس منتج. والـproductivity دائماً عملية لأنه **معدّل** (ناتج ÷ جهد).

Aggarwal, §1.5.4–1.5.5, pp.13–14.

1.8 · Effort is measured in terms of:

(a) Person-months · **(b)** Rupees · **(c)** Persons · **(d)** Months

Person-months (a) — Answer

"Most appropriate unit of effort is Person Months (PMs), meaning thereby number of persons involved for specified months." Option (c) counts people, option (d) counts time — effort is the **product** of the two, which is why neither alone works.

Aggarwal, §1.5.5, p.14.

1.9 · UML stands for

(a) Uniform modeling language · **(b)** Unified modeling language · **(c)** Unit modeling language · **(d)** Universal modeling language

Unified modeling language (b) — Answer

"Another definition from unified modeling language (UML) [OMG2K] is: 'A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.'"

Aggarwal, §1.5.6, p.14.

1.10 · An independently deliverable piece of functionality providing access to its services through interface is called

(a) Software measurement · **(b)** Software composition · **(c)** Software measure · **(d)** Software component

Software component (d) — Answer

"An independently deliverable piece of functionality providing access to its services through interfaces." "A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces."

Aggarwal, §1.5.6, p.14.

⚠ **هذا السؤال مطبوع `1.1` بسلايد 58 — رقم مكرر**

فهرسته هنا **1.10** حتى يمشي التسلسل 1.1 ← 1.20 بلا فراغ.

Aggarwal, §1.5.6, p.14.

1.11 · Infrastructure software are covered under

(a) Generic products · **(b)** Customized products · **(c)** Generic and Customized products · **(d)** None of the above

Generic products (a) — Answer

"Generic products are developed for anonymous customers... Infrastructure software like operating system, compilers, analyzers, word processors, CASE tools etc. are covered in this category."

Aggarwal, §1.5.7, p.14.

1.12 · Management of software development is dependent on

(a) people · **(b)** product · **(c)** process · **(d)** all of the above

all of the above (d) — Answer

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project." Options (a), (b) and (c) are each one of them.

Aggarwal, §1.6, p.15.

1.13 · During software development, which factor is most crucial?

(a) People · **(b)** Product · **(c)** Process · **(d)** Project

People (a) — Answer

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project." Order of dependency (Fig. 1.8): **People (1)** → **Product (2)** → **Process (3)** → **Project (4)**. People comes first, so it is the most crucial factor.

"The process priority is after people and product, however, it plays very critical role for the success of the project."

Aggarwal, §1.6, p.15; §1.6.3, p.16; Fig. 1.8, p.15.

1.14 · Program is

(a) subset of software · **(b)** super set of software · **(c)** software · **(d)** none of the above

subset of software (a) — Answer

Software = Program + Documentation + Operating Procedures.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.15 · Milestones are used to

(a) know the cost of the project · **(b)** know the status of the project · **(c)** know user expectations · **(d)** none of the above

know the status of the project (b) — Answer

"The milestones are the events that are used to ascertain the status of the project... The milestones are essential for project planning and management." Cost belongs to estimation, and user expectations belong to requirements — neither is what a milestone reports.

Aggarwal, §1.5.1, p.13.

1.16 · The term module used during design phase refers to

(a) Function · **(b)** Procedure · **(c)** Sub program · **(d)** All of the above

All of the above (d) — Answer

No single definition is fixed; a module may be **"a FORTRAN subroutine"**, **"an Ada package"**, **"procedures and functions of PASCAL and C"**, **"C++/Java classes"**, **"Java packages"**, or even **"a work assignment for an individual developer"** — **"all these definitions are correct."**

Aggarwal, §1.5.6, p.14.

1.17 · Software consists of

(a) Set of instructions + operating system · **(b)** Programs + documentation + operating procedures · **(c)** Programs + hardware manuals · **(d)** Set of programs

operating procedures + documentation + Programs (b) — Answer

Software = Program + Documentation + Operating Procedures (Fig. 1.1, p.4). Options (a), (c) and (d) each drop two of the three components.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.18 · Software engineering approach is used to achieve:

(a) Better performance of hardware · **(b)** Error free software · **(c)** Reusable software · **(d)** Quality software product

Quality software product (d) — Answer

Schach: **"a discipline whose aim is the production of** quality software**, software that is delivered on time, within budget, and that satisfies its requirements." "The art of programming only is no longer sufficient to construct large programs."**

Aggarwal, §1.2 and §1.2.1, p.4.

1.19 · Concept of software engineering are applicable to

(a) Fortran language only · **(b)** Pascal language only · **(c)** 'C' language only · **(d)** All of the above

All of the above (d) — Answer

The two definitions rest on **engineering principles** and on the **process** of producing quality software — they name no programming language, so the concepts are not tied to any one language.

Aggarwal, §1.2.1, p.4; Answers, p.498.

سبب اختيار (d)

التعريفان ما يذكران لغةً معينة؛ والمفتاح الرسمي يؤكد الخيار (d).

1.20 · CASE Tool is

(a) Computer Aided Software Engineering · **(b)** Component Aided Software Engineering · **(c)** Constructive Aided Software Engineering · **(d)** Computer Analysis Software Engineering

Computer Aided Software Engineering (a) — Answer

"Many people (especially CASE tool vendors) believe that CASE (Computer Aided Software Engineering) tools represent the so-called silver bullet..." "Operating systems, compilers, analysers, word processors, CASE tools etc."

Aggarwal, §1.1.2, p.3; §1.5.7, p.14.

Part B · Authored MCQs (same pattern)

B.1 · The definition of "software" differs from the everyday use of the word mainly because it includes:

(a) the hardware it runs on · **(b)** the documentation and operating procedures · **(c)** the development team · **(d)** the licence agreement

the documentation and operating procedures (b) — Answer

Software = Program + Documentation + Operating Procedures. The everyday sense of "software" is programs; the definition is a three-part construct, which is why a program alone is not a software product.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

B.2 · Which of the following belongs to the operating procedure manuals rather than to the documentation manuals?

(a) Design specification · **(b)** Data-flow diagram · **(c)** Installation guide · **(d)** Test results

Installation guide (c) — Answer

The two families are kept strictly apart. Design specifications, data-flow diagrams and test results are all **documentation**; the installation guide is listed among the **operating procedure** manuals.

Aggarwal, §1.2.2, pp.5–6 (Fig. 1.2, Fig. 1.3).

B.3 · A software house builds a payroll system for one specific ministry. Under the classification this product is:

(a) generic · **(b)** bespoke (customized) · **(c)** infrastructure · **(d)** open source

(customized) bespoke (b) — Answer

"The customized products are developed for particular customers." About 80% of development projects fall in this category.

Aggarwal, §1.5.7, p.14.

B.4 · Software process improvement commonly fails for all of the following reasons EXCEPT:

(a) not enough time · **(b)** lack of knowledge · **(c)** wrong motivations · **(d)** the absence of a CASE tool

the absence of a CASE tool (d) — Answer

Four reasons software process improvement is difficult: **"not enough time"**, **"lack of knowledge"**, **"wrong motivations"** and **"insufficient commitment"**. Tooling is not among them, so (d) is the exception.

Keep this separate from the different question of why CASE tools cannot end the crisis — that argument belongs to §1.1.2 and rests on the no-silver-bullet claim, not on this list. The two sections answer two different questions.

Aggarwal, §1.2.3, pp.6–7.

B.5 · Which statement is one of the developer-perspective software myths?

(a) The tested code is the only deliverable of a project · **(b)** A prototype reduces requirements risk · **(c)** Maintenance costs dominate the life-cycle cost · **(d)** Requirements change during development

The tested code is the only deliverable of a project (a) — Answer

Developer-perspective myth: **"Tested code is only one of the deliverable!"** Options (b), (c) and (d) are supported positions, not myths.

Aggarwal companion slides, slide 45 (Software Myths — Developer Perspectives).

B.6 · The aim of software development has shifted from producing working programs to producing:

(a) smaller programs · **(b)** good quality maintainable programs · **(c)** programs with more features · **(d)** error-free programs

good quality maintainable programs (b) — Answer

"The aim has been shifted from developing working programs to good quality, maintainable programs." The change was driven by the **"cost of maintaining software"**.

Aggarwal, §1.4, p.12; §1.2.1, p.4.

B.7 · "The strength and stability of a bridge can be calculated with reasonable precision" is listed as a difference between building a bridge and writing a program. The corresponding statement for a program is:

(a) the correctness of a program can be calculated with existing methods · **(b)** it is not possible to calculate the correctness of a program with existing methods · **(c)** every program is a repeat of a previous program · **(d)** the requirements for a program do not change during construction

program with existing it is not possible to calculate the correctness of a (b) — Answer methods

The bridge-versus-program comparison pairs every engineering strength with a software weakness: the bridge problem is **well understood**, the program problem is **only partly understood**; bridge requirements **barely change**, program requirements **change in all phases**; bridges are **calculable**, programs are **not**.

Aggarwal, §1.2.4, Table 1.1, p.8.

B.8 · If productivity is defined as output per unit of effort, and effort is measured in person-months, then productivity is measured in:

(a) lines of code · **(b)** person-months · **(c)** lines of code per person-month · **(d)** person-months per line of code

lines of code per person-month (c) — Answer

"Productivity may be measured as LOC/PM (lines of code produced/person month)." Option (d) inverts the ratio — a classic trap, because it is the **effort per unit of output** (the inverse rate), not productivity.

Aggarwal, §1.5.5, p.14.

ASE CHAPTER 2

T1 MCQ Question Pack · Life-Cycle Models

Question 2.1

Spiral Model was developed by: - (a) Bev Littlewood - (b) Barry Boehm - (c) Roger Pressman - (d) Victor Basili

(b) Barry Boehm

Technical Rationale: The spiral model was formulated and published by Barry Boehm in 1986 to incorporate risk analysis and management into the software development process.

ملاحظة الطالب والفخ الامتحاني: - باري بوم (Barry Boehm) هو صاحب فكرة النموذج الحلزوني ونموذج تقدير التكاليف الشهير COCOMO. احذر الخلط مع روجر بريسمان (مؤلف الكتب) أو فيكتور باسيلي. (تنبيه: ورد اسمه مطبوعاً في الكتاب الأصلي ص. 37 والشريحة 41 بالخطأ المطبعي "Berry Boehm"، والمفتاح ص. 499 يعتمد الخيار (b)، والصحيح علمياً هو Barry Boehm).

Source: K.K. Aggarwal & Singh (2007), §2.4, p. 27; Printed Question p. 37 (Q2.1) & Slide 41; Official Answer Key p.499.

Question 2.2

Which model is most popular for student's small projects? - (a) Waterfall model - (b) Spiral model - (c) Quick and fix model - (d) Prototyping model

(c) Quick and fix model

Technical Rationale: The quick and fix model (also referred to as Build & Fix) is the most popular approach for small student programming exercises of 100 to 200 lines because code is written immediately without requiring formal specifications or architectural design.

ملاحظة الطالب والفخ الامتحاني: - يُطلق عليه في مراجع أخرى Build and Fix؛ هو أسلوب عفوي (Ad-hoc) يناسب التمارين الفردية الصغيرة ولا يصلح للمشاريع الحقيقية.

Source: K.K. Aggarwal & Singh (2007) Slides 4–5; Slide 41 (Q2.2).

Question 2.3

Which is not a software life cycle model? - (a) Waterfall model - (b) Spiral model - (c) Prototyping model - (d) Capability maturity model

(d) Capability maturity model

Technical Rationale: Waterfall, Spiral, and Prototyping are software development process/life-cycle models defining phases of development. The Capability Maturity Model (CMM) is an organizational process improvement and maturity appraisal framework developed by the SEI, not an SDLC process model.

ملاحظة الطالب والفخ الامتحاني: - نموذج CMM يقيّم نضج المنظمة البرمجية ككل (Maturity Framework)، ولا يحدد مراحل بناء البرمجيات كدورة حياة.

Source: K.K. Aggarwal & Singh (2007) Slide 41 (Q2.3).

Question 2.4

Project risk factor is considered in: - (a) Waterfall model - (b) Prototyping model - (c) Spiral model - (d) Iterative enhancement model

(c) Spiral model

Technical Rationale: The primary defining feature of the spiral model is explicit, structured risk management. Quadrant 2 of each 360-degree loop is specifically dedicated to identifying, analyzing, and resolving technical and management risks.

Source: K.K. Aggarwal & Singh (2007) Slide 19, 21; Slide 41 (Q2.4).

Question 2.5

SDLC stands for: - (a) Software design life cycle - (b) Software development life cycle - (c) System development life cycle - (d) System design life cycle

(b) Software development life cycle

Technical Rationale: Within software engineering, SDLC standardly denotes the Software Development Life Cycle, representing the structured phases from software conception to delivery and maintenance.

Source: K.K. Aggarwal & Singh (2007) Slide 3; Slide 41 (Q2.5).

Question 2.6

Build and fix model has: - (a) 3 phases - (b) 1 phase - (c) 2 phases - (d) 4 phases

(c) 2 phases

Technical Rationale: The Build & Fix model is defined as a simple two-phase ad-hoc model: write code (Build), and then modify code to repair errors (Fix).

Source: K.K. Aggarwal & Singh (2007) Slide 4; Slide 42 (Q2.6).

Question 2.7

SRS stands for: - (a) Software requirements specification - (b) Software requirements solution - (c) System requirements specification - (d) None of the above

(a) Software requirements specification

Technical Rationale: SRS standardly stands for Software Requirements Specification, the formal document produced at the conclusion of requirements analysis detailing what a system will do.

> **ملاحظة الطالب والفخ الامتحاني:** - الـSRS هي الوثيقة التعاقدية الرسمية التي تصف "ماذا" يفعل النظام، وتثبت المتطلبات الوظيفية وغير الوظيفية.

Source: B.B. Agarwal et al. (2010), §3.2.2, p. 57; K.K. Aggarwal & Singh (2007) Slide 10, 42 (Q2.7).

Question 2.8

Waterfall model is not suitable for: - (a) Small projects - (b) Accommodating change - (c) Complex projects - (d) None of the above

(b) Accommodating change

Technical Rationale: A major recognized weakness of the classical waterfall model is that requirements are frozen early and development proceeds strictly linearly, making it unsuitable for accommodating frequent changes during project execution.

> **ملاحظة الطالب والفخ الامتحاني:** - السلايد 8 ينص حرفياً: "This model is not suitable for accommodating any change". كما أنه ضعيف في المشاريع الكبيرة، لكن عدم استيعاب التغيير هو العيب المنهجي الأبرز.

Source: K.K. Aggarwal & Singh (2007) Slide 8; Slide 42 (Q2.8).

Question 2.9

RAD stands for: - (a) Rapid application development - (b) Relative application development - (c) Ready application development - (d) Repeated application development

(a) Rapid application development

Technical Rationale: RAD denotes Rapid Application Development, a model designed to reduce development time through rapid prototyping, active user participation, and component reuse.

Source: K.K. Aggarwal & Singh (2007) Slide 12; Slide 42 (Q2.9).

Question 2.10

RAD model was proposed by: - (a) Lucent Technologies - (b) Motorola - (c) IBM - (d) Microsoft

(c) IBM

Technical Rationale: The Rapid Application Development model was developed by IBM in 1980.

> **ملاحظة الطالب والفخ الامتحاني:** - تنص شريحة المصدر 12 بوضوح: "Developed by IBM in 1980".

Source: K.K. Aggarwal & Singh (2007) Slide 12; Slide 42 (Q2.10).

Question 2.11

If requirements are easily understandable and defined, which model is best suited? - (a) Waterfall model - (b) Prototyping model - (c) Spiral model - (d) None of the above

(a) Waterfall model

Technical Rationale: When requirements are completely understood, stable, and clearly defined upfront, the linear sequential waterfall model is best suited because it offers clean, disciplined phase execution.

> **ملاحظة الطالب والفخ الامتحاني:** - مصفوفة متطلبات الشريحة 37: "Are requirements easily understandable and defined? → Waterfall: Yes, RAD: Yes". الخيار الوحيد هنا بين النماذج المطروحة هو Waterfall.

Source: K.K. Aggarwal & Singh (2007) Slide 37; Slide 43 (Q2.11).

Question 2.12

If requirements are frequently changing, which model is to be selected? - (a) Waterfall model - (b) Prototyping model - (c) RAD model - (d) Iterative enhancement model

(b) Prototyping model

Technical Rationale: When requirements change frequently or are unstable, the prototyping model is selected to construct rapid throw-away versions that allow users to evaluate features and stabilize requirements before full development.

> **ملاحظة الطالب والفخ الامتحاني:** - في مصفوفة الشريحة 37: "Do we change requirements quite often? → Prototype: Yes ,Spiral: Yes". بما أن Spiral غير موجود في الخيارات، فالجواب هو Prototyping model.

Source: K.K. Aggarwal & Singh (2007) Slide 37; Slide 43 (Q2.12).

Question 2.13

If user participation is available, which model is to be chosen? - (a) Waterfall model - (b) Iterative enhancement model - (c) Spiral model - (d) RAD model

(d) RAD model

Technical Rationale: The RAD model strictly mandates active and continuous user involvement across all phases (Joint Application Development) to evaluate progress and guide component construction.

> **ملاحظة الطالب والفخ الامتحاني:** - الشريحة 39 صريحة: "User involvement in all phases → Prototype: Yes ,RAD: Yes". بين الخيارات المعروضة، RAD هو الأنسب للاشتراك المكثف.

Source: K.K. Aggarwal & Singh (2007) Slide 12, 39; Slide 43 (Q2.13).

Question 2.14

If limited user participation is available, which model is to be selected? - (a) Waterfall model - (b) Spiral model - (c) Iterative enhancement model - (d) Any of the above

(d) Any of the above

Technical Rationale: According to the selection matrix based on user involvement, Waterfall, Spiral, and Iterative Enhancement all permit execution under limited user participation (unlike RAD, which strictly fails without heavy user involvement).

> **ملاحظة الطالب والفخ الامتحاني:** - الشريحة 39: سطر "Limited user participation" يضع علامة Yes أمام Waterfall و Iterative enhancement و Evolutionary و Spiral، ويضع No أمام RAD و Prototype. لذلك الخيار الصحيح هو "any of the above".

Source: K.K. Aggarwal & Singh (2007) Slide 39; Slide 43 (Q2.14).

Question 2.15

If project is the enhancement of existing system, which model is best suited? - (a) Waterfall model - (b) Prototyping model - (c) Iterative enhancement model - (d) Spiral model

(c) Iterative enhancement model

Technical Rationale: When a project represents an upgrade or enhancement to an existing, already operational software system, the iterative enhancement model is designed specifically to deliver added capabilities across successive increments.

> **ملاحظة الطالب والفخ الامتحاني:** - الشريحة 40: "Iterative → Project is the enhancement of the existing system" :enhancement ,Yes :Evolutionary ,Yes :RAD ,Yes". بين الخيارات المتاحة، Iterative enhancement هو الخيار المطابق.

Source: K.K. Aggarwal & Singh (2007) Slide 40; Slide 43 (Q2.15).

Question 2.16

Which one is the most important feature of spiral model? - (a) Quality management - (b) Risk management - (c) Performance management - (d) Efficiency management

(b) Risk management

Technical Rationale: Risk management is the foundational, indispensable characteristic of Boehm's spiral model; the process is explicitly designed to identify and mitigate risks in every cycle before incurring further costs.

Source: K.K. Aggarwal & Singh (2007) Slide 19, 21; Slide 44 (Q2.16).

Question 2.17

Most suitable model for new technology that is not well understood is: - (a) Waterfall model - (b) RAD model - (c) Iterative enhancement model - (d) Evolutionary development model

(d) Evolutionary development model

Technical Rationale: The evolutionary development model is recommended when a project utilizes novel or poorly understood technology, as it enables developers to explore technical feasibility iteratively and evolve functionality by category.

> **ملاحظة الطالب والفخ الامتحاني:** - نص الشريحة 15: "technology that This model is useful for projects using new is not well understood." احذر اختيار RAD لأن RAD يفشل إذا كانت التكنولوجيا جديدة وغير مجربة.

Source: K.K. Aggarwal & Singh (2007) Slide 15; Slide 44 (Q2.17).

Question 2.18

Statistically, the maximum percentage of errors belong to the following phase of SDLC: - (a) Coding - (b) Design - (c) Specifications - (d) Installation and maintenance

(c) Specifications

> **ملاحظة الطالب والفخ الامتحاني:** - مفتاح الكتاب الرسمي (ص. 499) والشريحة 44 يحددان الخيار (c) كإجابة معتمدة للامتحان. - **حدود الدليل:** لا يورد متن الفصل أي جدول إحصائي أو مسح ميداني يثبت نسب الأخطاء بين المراحل؛ لذا يُحفظ المفتاح للامتحان دون اعتباره حقيقة إحصائية مثبتة في المتن.

Source: K.K. Aggarwal & Singh (2007) Slide 44 (Q2.18); Official Key p. 499.

Question 2.19

Which phase is not available in software life cycle? - (a) Coding - (b) Testing - (c) Maintenance - (d) Abstraction

(d) Abstraction

Technical Rationale: Coding, Testing, and Maintenance are canonical operational phases of software life cycles. Abstraction is a fundamental software engineering principle and mental technique, not a designated chronological SDLC phase.

> **ملاحظة الطالب والفخ الامتحاني:** - التجريد (Abstraction) مفهوم تصميمي وهندسي يُمارس في التحليل والتصميم، وليس مرحلة زمنية مستقلة في دورة الحياة.

Source: K.K. Aggarwal & Singh (2007) Slide 3; Slide 44 (Q2.19).

Question 2.20

The development is supposed to proceed linearly through the phases in: - (a) Spiral model - (b) Waterfall model - (c) Prototyping model - (d) None of the above

(b) Waterfall model

Technical Rationale: The waterfall model is inherently structured as a linear-sequential process where development is assumed to flow in a strict, unbranching cascade from one phase to the next.

Source: B.B. Agarwal et al. (2010), §2.2, p. 36; K.K. Aggarwal & Singh (2007) Slide 6; Slide 44 (Q2.20).

Question 2.21

Unified process is maintained by: - (a) Infosys - (b) Rational software corporation - (c) SUN Microsystems - (d) None of the above

(b) Rational software corporation

Technical Rationale: The Unified Process was enhanced, maintained, and productized by the Rational Software Corporation (which was subsequently acquired by IBM), and is commonly known as the Rational Unified Process (RUP).

Source: K.K. Aggarwal & Singh (2007) Slide 23; Slide 45 (Q2.21).

Question 2.22

Unified process is: - (a) Iterative - (b) Incremental - (c) Evolutionary - (d) All of the above

(d) All of the above

Technical Rationale: The Unified Process integrates multiple paradigms: it is iterative (structured as successive iterations), incremental (delivering expanded capability cycle by cycle), and evolutionary (advancing the architectural baseline over time).

ملاحظة الطالب والفخ الامتحاني: - يتميز الـProcess Unified بأنه يجمع الخصائص الثلاث معاً: Iterative و Incremental و Evolutionary، لذلك الجواب هو All of the above.

Source: K.K. Aggarwal & Singh (2007) Slide 23, 26; Slide 45 (Q2.22).

Question 2.23

Who is not in the team of Unified process development? - (a) I. Jacobson - (b) G. Booch - (c) B. Boehm - (d) J. Rumbaugh

(c) B. Boehm

Technical Rationale: The Unified Process and UML were created by the "Three Amigos": Ivar Jacobson, Grady Booch, and James Rumbaugh. Barry Boehm is the creator of the Spiral Model and COCOMO, not an author of the Unified Process.

ملاحظة الطالب والفخ الامتحاني: - نص السلايد 23 يذكر الثلاثي: "Developed by I. Jacobson, G. Booch and J. Rumbaugh". باري بوم (Boehm) هو رائد النموذج الحلزوني وليس من مبتكري UP.

Source: K.K. Aggarwal & Singh (2007) Slide 23; Slide 45 (Q2.23).

Question 2.24

How many phases are in the unified process? - (a) 4 - (b) 5 - (c) 2 - (d) None of the above

(a) 4

Technical Rationale: The Unified Process divides the life cycle into exactly four chronological phases: Inception, Elaboration, Construction, and Transition.

ملاحظة الطالب والفخ الامتحاني: - عدد مراحل UP هو 4 حصراً: Inception ← Elaboration ← Construction ← Transition.

Source: K.K. Aggarwal & Singh (2007) Slide 24; Slide 45 (Q2.24).

Question 2.25

The outcome of construction phase can be treated as: - (a) Product release - (b) Beta release - (c) Alpha release - (d) All of the above

(b) Beta release

Technical Rationale: The milestone of the Construction phase is Initial Operational Capability (IOC), producing an operational software product release distributed for beta testing and user evaluation prior to final transition.

ملاحظة الطالب والفخ الامتحاني: - مرحلة Transition تبدأ باختبارات البيتا (Beta testing)، مما يعني أن ناتج مرحلة Construction هو الإصدار التجريبي المكتمل وظيفياً (Beta release). بينما الإصدار النهائي المكتمل للمستهلك (Product release) هو ناتج Transition.

Source: K.K. Aggarwal & Singh (2007) Slides 32–35; Slide 45 (Q2.25).

Retrieval set — Focused Active Recall Drill

1 **What fundamental distinction separates an SDLC process model from an organizational framework like CMM?**

An SDLC process model (such as Waterfall, Spiral, or RAD) prescribes the chronological phases, technical activities, and deliverables for engineering software. In contrast, the Capability Maturity Model (CMM) is an organizational appraisal framework that assesses the maturity and process management capability of the organization as a whole, rather than governing development phases.

Source: K.K. Aggarwal & Singh (2007) Slide 41 (Q2.3).

2 **Why does the Waterfall model require requirements to be completely defined and frozen before design and coding commence?**

Because development is structured to proceed strictly linearly without backward iteration. Once a phase is completed, subsequent stages depend entirely on its baseline deliverables, making late requirement changes disruptive and difficult to accommodate within a linear cascade.

Source: K.K. Aggarwal & Singh (2007) Slides 6, 8, 9; Slide 42 (Q2.8); Slide 44 (Q2.20).

ASE CHAPTER 3

T1 Multi Choice Questions Pack · SW Requirements

Section 1: Publisher Companion Slide MCQs (25 Items)

Q3.1. Which one is not a step of requirement engineering?

(a) Requirements elicitation · (b) Requirements analysis · (c) Requirements design · (d) Requirements documentation

> **(c) Requirements design**
>
> **Requirements engineering consists of four crucial process steps: requirements elicitation, requirements analysis, requirements documentation (producing the SRS), and requirements review (validation).** "Requirements design" is not a recognized phase; software design is a distinct subsequent phase of the life cycle that operates upon the frozen SRS.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, p. 41; Slide 121. [Independently Checked]*

Q3.2. Requirements elicitation means:

(a) Gathering of requirements · (b) Capturing of requirements · (c) Understanding of requirements · (d) All of the above

> **(d) All of the above**
>
> **Requirements elicitation encompasses discovering, gathering, capturing, and understanding the real needs of customers, users, and organizational stakeholders.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4, p. 49; Slide 121. [Independently Checked]*
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الاستخلاص ليس مجرد "جمع" سلبي للمعلومات، بل يشمل الاستكشاف والتوضيح والاستيعاب العميق لرغبات العميل والقيود المحيطة بالنظام.

Q3.3. SRS stands for:

(a) Software requirements specification · (b) System requirements specification · (c) Systematic requirements specifications · (d) None of the above

> **(a) Software requirements specification**
>
> **SRS stands for Software Requirements Specification.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, p. 72; Slide 122. [Independently Checked]*

Q3.4. SRS document is for:

(a) "What" of a system? · (b) How to design the system? · (c) Costing and scheduling of a system · (d) System's requirement.

(a) "What" of a system?

The SRS describes what the software must do, including required external behavior and interfaces, while leaving internal design and implementation decisions to the design stage.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, p. 40; Slide 122. [Independently Checked]

Q3.5. Requirements review process is carried out to:

(a) Spend time in requirements gathering · (b) Improve the quality of SRS · (c) Document the requirements · (d) None of the above

(b) Improve the quality of SRS

The requirements review process is conducted to detect and correct omissions, ambiguities, contradictions, and unrealizable requirements, thereby directly improving the technical quality and baseline integrity of the SRS.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, p. 41; Slide 123. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- مراجعة المتطلبات (Requirements Review) هي نشاط توكيد جودة يهدف لكشف العيوب في الـ SRS قبل الانتقال للتصميم لتفادي كلفة الإصلاح المضاعفة لاحقاً.

Q3.6. Which one of the statements is not correct during requirements engineering?

(a) Requirements are difficult to uncover · (b) Requirements are subject to change · (c) Requirements should be consistent · (d) Requirements are always precisely known.

(d) Requirements are always precisely known.

In empirical software engineering practice, requirements are rarely precisely known upfront; customers often have vague initial concepts, requirements constantly change, and discovering complete needs requires disciplined iteration.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–41; Slide 123. [Independently Checked]

Q3.7. Which one is not a type of requirements?

(a) Known requirements · (b) Unknown requirements · (c) Undreamt requirements · (d) Complex requirements

(d) Complex requirements

The textbook categorizes requirements discovery into three states: known requirements (explicitly understood and stated), unknown requirements (revealed during investigation), and undreamt requirements (innovative features beyond user expectations). "Complex requirements" is not a recognized classification category in this model.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2, p. 45; Slide 124. [Independently Checked]

Q3.8. Which one is not a requirements elicitation technique?

(a) Interviews · (b) The use case approach · (c) FAST · (d) Data flow diagram.

(d) Data flow diagram.

Interviews, the use case approach, and FAST are requirements elicitation techniques used to capture user needs. A Data Flow Diagram (DFD) is a structural requirements analysis and data-modeling tool, not an elicitation technique.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4, p. 49 & §3.5.1, p. 60; Slide 124. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- ميز بين أدوات الاستخلاص (Interviews, Use Cases, FAST, QFD) وأدوات التحليل والنمذجة الهيكلية (DFD, ERD, Data Dictionary).

Q3.9. FAST stands for:

(a) Functional Application Specification Technique · (b) Fast Application Specification Technique · (c) Facilitated Application Specification Technique · (d) None of the above

(c) Facilitated Application Specification Technique

FAST stands for Facilitated Application Specification Technique.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.3, p. 51; Slide 125. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- كلمة FAST تعني Facilitated (ميسّرة)، وهي جلسات عمل مشتركة ومكثفة بين العملاء والمطورين يقودها ميسر محايد.

Q3.10. QFD in requirement engineering stands for:

(a) Quality function design · (b) Quality factor design · (c) Quality function development · (d) Quality function deployment

(d) Quality function deployment

QFD stands for Quality Function Deployment.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, p. 53; Slide 125. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- احفظ المصطلح بدقة: Deployment (نشر/تطبيق وظيفة الجودة)، وترجمتها لمتطلبات هندسية ملموسة.

Q3.11. Which is not a type of requirements under quality function deployment:

(a) Normal requirements · (b) Abnormal requirements · (c) Expected requirements · (d) Exciting requirements

(b) Abnormal requirements

Under Quality Function Deployment (QFD), requirements are structured into Normal requirements (stated objectives), Expected requirements (implicit, baseline requirements), and Exciting requirements (innovative features that delight the customer). "Abnormal requirements" is not a QFD category.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, p. 53; Slide 126. [Independently Checked]

Q3.12. Use case approach was developed by:

(a) I. Jacobson and others · (b) J.D. Musa and others · (c) B. Littlewood · (d) None of the above

> **(a) I. Jacobson and others**
>
> **The Use Case Approach was developed by Ivar Jacobson and his colleagues (1992) as a scenario-driven method for capturing system behavior from the viewpoint of external actors.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, p. 54; Slide 126. [Independently Checked]*

Q3.13. Context diagram explains:

(a) The overview of the system · (b) The internal view of the system · (c) The entities of the system · (d) None of the above

> **(a) The overview of the system**
>
> **A context diagram represents the highest-level structural overview of the system, abstracting the entire system as a single process bubble interacting with external environment entities.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5, p. 59; Slide 127. [Independently Checked]*

Q3.14. DFD stands for:

(a) Data Flow design · (b) Descriptive functional design · (c) Data flow diagram · (d) None of the above

> **(c) Data flow diagram**
>
> **DFD stands for Data Flow Diagram.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 60; Slide 127. [Independently Checked]*

Q3.15. Level-0 DFD is similar to:

(a) Use case diagram · (b) Context diagram · (c) System diagram · (d) None of the above

> **(b) Context diagram**
>
> **In structured analysis, a Level-0 DFD is synonymous with the Context Diagram, modeling the whole target application as single process 0.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 61; Slide 128. [Independently Checked]*

Q3.16. ERD stands for:

(a) Entity relationship diagram · (b) Exit related diagram · (c) Entity relationship design · (d) Exit related design

> **(a) Entity relationship diagram**
>
> **ERD stands for Entity Relationship Diagram.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, p. 63; Slide 128. [Independently Checked]*

Q3.17. Which is not a characteristic of a good SRS?

(a) Correct · (b) Complete · (c) Consistent · (d) Brief

(d) Brief

IEEE Standard 830-1998 specifies that a good SRS must be correct, unambiguous, complete, consistent, ranked for importance/stability, verifiable, modifiable, and traceable. Brevity is not a standard quality criterion; striving for shortness frequently introduces omissions and ambiguities.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, p. 73; Slide 129. [Independently Checked]

Q3.18. Outcome of requirements specification phase is:

(a) Design Document · (b) Software requirements specification · (c) Test Document · (d) None of the above

(b) Software requirements specification

The deliverable produced by the requirements specification phase is the Software Requirements Specification (SRS).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, p. 72; Slide 129. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- وثيقة SRS هي الناتج الرسمي لمرحلة المواصفات، وهي العقد الفاصل بين متطلبات المستخدم وبداية التصميم المعماري.

Q3.19. The basic concepts of ER model are:

(a) Entity and relationship · (b) Relationships and keys · (c) Entity, effects and relationship · (d) Entity, relationship and attribute

(d) Entity, relationship and attribute

The three foundational modeling concepts of the Entity-Relationship (ER) model are Entities (data objects), Relationships (associations among entities), and Attributes (properties qualifying entities or relationships).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, p. 63; Slide 130. [Independently Checked]

Q3.20. The DFD depicts:

(a) Flow of data · (b) Flow of control · (c) Both (a) and (b) · (d) None of the above

(a) Flow of data

A Data Flow Diagram depicts solely the flow and transformation of data through system processes; it contains no control flow, decision sequence, or loop logic.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 60; Slide 130. [Independently Checked]

Q3.21. Product features are related to:

(a) Functional requirements · (b) Non functional requirements · (c) Interface requirement · (d) None of the above

(a) Functional requirements

Product features describe the operational capabilities, services, and functional transformations provided by the software to its end users, defining functional requirements.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 131. [Independently Checked]

Q3.22. Which one is a quality attribute?

(a) Reliability · (b) Availability · (c) Security · (d) All of the above

(d) All of the above

Reliability, availability, and security are examples of non-functional quality attributes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 131. [Independently Checked]

Q3.23. IEEE standard for SRS is:

(a) IEEE Standard 837-1998 · (b) IEEE Standard 830-1998 · (c) IEEE Standard 832-1998 · (d) IEEE Standard 839-1998

(b) IEEE Standard 830-1998

The canonical standard for Software Requirements Specifications established in course literature is IEEE Standard 830-1998 (Recommended Practice for Software Requirements Specifications).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.2, p. 46; Slide 132. [Independently Checked]

Q3.24. Which one is not a functional requirement?

(a) Efficiency · (b) Reliability · (c) Product features · (d) Stability

No unique answer: (a) Efficiency, (b) Reliability, and (d) Stability are all non-functional requirements.

Product features represent functional requirements, whereas efficiency, reliability, and stability are non-functional quality attributes. In this publisher slide item (Slide 132), the question contains a recognized structural flaw in its negative phrasing: three options ((a) Efficiency, (b) Reliability, (d) Stability) are non-functional requirements, while only (c) Product features is functional. If the examiner asks "*Which one is a functional requirement?*", the intended answer is (c). If asked verbatim as printed, any of (a), (b), or (d) is technically correct, with Reliability and Efficiency being canonical textbook quality attributes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 132. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- **تنبيه لعيب صياغة المصدر:** هذا السؤال ورد نصاً بالسلايد 132 بصياغة ملتبسة؛ فخيارات (Efficiency, Reliability, Stability) كلها متطلبات غير وظيفية (Non-Functional / Quality Attributes)، بينما (Product features) هو الوحيد الوظيفي. إذا جاء السؤال كما هو، فإن (a) أو (b) أو (d) صحيحة بالمعنى الحرفي، أما إذا عُدلت الصياغة في الامتحان إلى "*Which one IS a functional requirement?*" فالجواب هو قطعاً (c).

MCQ-25 (Slide 133, printed as duplicate 3.23). APIs stand for:

(a) Application performance interfaces · (b) Application programming interfaces · (c) Application programming integration · (d) Application performance integration

(b) Application programming interfaces

APIs stand for Application Programming Interfaces, which establish procedural interface specifications for programmatic inter-software communication.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.3, p. 46; Slide 133. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- API = Application Programming Interface، وهو يمثل نمطاً للمواصفات الإجرائية (Procedural Interface).
  الرقم طُبع مكرراً كـ 3.23 بالسلايد 133 وثبتناه كـ MCQ-25.

Section 2: Additional Chapter 3 Synthesis MCQs (10 Items)

Q3.26. What are the three primary types of interface specifications required in software engineering?

(a) User interface, Database interface, and Network interface · (b) Procedural interfaces (APIs), Data structures, and Representation of data · (c) Hardware interface, Firmware interface, and Software interface · (d) Input interface, Output interface, and Storage interface

(b) Procedural interfaces (APIs), Data structures, and Representation of data

The three types of interface specifications are procedural interfaces, data structures, and representation of data.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.3, p. 46; Slide 9.

ملاحظة الطالب والفخ الامتحاني:

- التزم حرفياً بتبويب الشريحة 9 المعتمدة للدكتور: الأنواع الثلاثة لمواصفات الواجهات هي: الواجهات الإجرائية (Procedural interfaces - APIs)، هياكل البيانات (Data structures)، وتمثيل البيانات (Representation of data).
  لا تخلط بين هذا التبويب الخاص بالسلايد وبين تبويب معيار IEEE 830 الخارجي (User, Hardware, Software, Communications).

Q3.27. Under a Feasibility Study, which dimension evaluates whether the development organization has the necessary tools, platforms, and technical expertise?

(a) Operational feasibility · (b) Economic feasibility · (c) Technical feasibility · (d) Schedule feasibility

(c) Technical feasibility

Technical feasibility determines whether the proposed application can be constructed using available technology, whether technical constraints can be satisfied, and whether the development team possesses the requisite technical competencies.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3.1, pp. 47–48; Slides 11–13.

Q3.28. What is the bounded objective of risk identification within the Feasibility Study phase according to Chapter 3 lecture materials?

(a) Executing full spiral risk cycles · (b) Determining whether major project risks are identified and can be successfully surmounted · (c) Eliminating all project uncertainties permanently · (d) Purchasing commercial insurance policies

(b) Determining whether major project risks are identified and can be successfully surmounted

Within the feasibility study, the focus is on determining whether major project risks have been identified and whether they can be surmounted before the project proceeds.

Source: Chapter 3 lecture, Slide 14; Dr. Ali Fahim Vocabulary Ledger (2026-09-30).

ملاحظة الطالب والفخ الامتحاني:

- انتبه لحدود إدارة المخاطر في هذا الفصل: ليست دورة حلزونية كاملة، بل التحقق من أن المخاطر الجوهرية محددة وقابلة للتجاوز (Surmountable) كشرط للمضي بالمشروع.

Q3.29. How do User Requirements fundamentally differ from System Requirements?

(a) User requirements are written in code; system requirements are written in English · (b) User requirements state high-level customer expectations in natural language and diagrams, while system requirements define structured, detailed functional and operational specifications for engineers · (c) User requirements describe non-functional constraints only; system requirements describe functions only · (d) User requirements are authored during testing; system requirements during elicitation

(b) User requirements state high-level customer expectations in natural language and diagrams, while system requirements define structured, detailed functional and operational specifications for engineers

User requirements are high-level statements in natural language and informal diagrams defining what services the system provides and its operational constraints for clients and management. System requirements are comprehensive, structured descriptions of software functions, services, and operational constraints serving as the contract and basis for architectural design.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Slide 8; Sommerville (9th ed.), §4.1, pp. 84–85.

Q3.30. In Data Flow Diagramming (DFD), what constitutes a "Black Hole" defect?

(a) A process that produces outputs without receiving any inputs · (b) A process that has input data flows but generates no output data flows · (c) A data store that is never accessed · (d) A data flow connecting two external entities

(b) A process that has input data flows but generates no output data flows

In DFD rule verification, a "Black Hole" refers to a process that consumes input data flows but produces zero output data flows, violating the fundamental principle of data transformation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 60–61; B.B. Agarwal et al. (2010), §3.3.1, p. 57.

ملاحظة الطالب والفخ الامتحاني:

- الثقب الأسود (Black Hole) في الـ DFD: عملية تدخلها بيانات دون أن يخرج منها أي تدفق. عكسها المعجزة (Miracle): عملية تنتج مخرجات دون أي مدخلات!

Q3.31. Which data flow connection is strictly illegal in a valid Data Flow Diagram?

(a) Process to Process · (b) Process to Data Store · (c) Data Store directly to another Data Store without an intervening Process · (d) External Entity to Process

(c) Data Store directly to another Data Store without an intervening Process

Data cannot move autonomously between data stores or between external entities; every valid data flow in a DFD must have a Process as either its origin or its destination.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 60–61; Slides 85–86.

Q3.32. In the course Data Dictionary notation, what does { } indicate?

(a) An optional data element · (b) Selection of one alternative from a list · (c) Repetition of a data element, with adjacent bounds specifying its permitted count · (d) Sequential composition of data elements

(c) Repetition of a data element, with adjacent bounds specifying its permitted count

In Table 3.3, x=a+b means x consists of a and b; x=[a/b] means either a or b; and x=(a) marks an optional item. Braces denote repetition: x=y{a}z means between y and z occurrences of a, while x=y{a} and x={a}z express lower-only and upper-only bounds, respectively.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, Table 3.3, p. 63; Slide 88; B.B. Agarwal et al. (2010), §3.3.2, p. 58.

ملاحظة الطالب والفخ الامتحاني:

- تنبيه مصدري: جدول 3.3 في كتاب أغروال يكتب الاختيار بين البدائل هكذا x=[a/b]، بينما سلايد 88 يستخدم {a/b}؛ اعتمدنا صيغة الكتاب لأنه المصدر الأساسي. أما التكرار المحدود في الكتاب فيظهر بصيغة x=y{a}z.

Q3.33. A requirement states: "*The software shall be intuitive, fast, and extremely user-friendly.*" Which IEEE 830 characteristic is directly violated?

(a) Modifiability · (b) Verifiability · (c) Traceability · (d) Ranking for importance

(b) Verifiability

A requirement is verifiable if and only if there exists a finite, cost-effective process by which a person or machine can measure whether the delivered software meets the requirement. Subjective adjectives like "intuitive", "fast", and "user-friendly" cannot be objectively verified or tested without quantifiable metrics (such as task completion time or maximum click count).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 74–75; Slide 104.

Q3.34. In Use Case modeling, what is the defining distinction between an <<include>> and an <<extend>> relationship?

(a) <<include>> is optional; <<extend>> is mandatory · (b) <<include>> unconditionally executes shared common behavior, while <<extend>> conditionally adds optional behavior at defined extension points · (c) <<include>> connects actors only; <<extend>> connects use cases only · (d) They are functionally identical

(b) <<include>> unconditionally executes shared common behavior, while <<extend>> conditionally adds optional behavior at defined extension points

An <<include>> relationship represents mandatory factoring of common subroutines that execute unconditionally during the base use case. An <<extend>> relationship represents optional or exceptional behavior that is executed only if specific trigger conditions are met at declared extension points.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, p. 54; Slides 30–33.

Q3.35. What is the fundamental difference between Requirements Verification and Requirements Validation?

(a) Verification checks code syntax; Validation checks compiler optimization · (b) Verification asks "Are we building the product right?" (conformance to specification), while Validation asks "Are we building the right product?" (satisfying actual customer needs) · (c) Verification is performed exclusively by end users; Validation is performed by software developers · (d) Verification is dynamic testing; Validation is static review

(b) Verification asks "Are we building the product right?" (conformance to specification), while Validation asks "Are we building the right product?" (satisfying actual customer needs)

Verification ensures that intermediate engineering artifacts strictly adhere to preceding lifecycle specifications (conformance). Validation ensures that the specified system accurately satisfies the real-world operational intentions and operational objectives of the customer.

Source: Slide 111; B.B. Agarwal et al. (2010), §3.8, p. 68.

ASE CHAPTER 4

Multiple Choice Questions · SW Project Planning

Q4.1. After the finalization of SRS, we may like to estimate:

(a) Size · (b) Cost · (c) Development time · (d) All of the above

(d) All of the above

Software project planning begins once the SRS is baseline-approved; the primary estimation objectives are software size, project cost/effort, development schedule/time, and required staffing. These estimates are related: size is an input to effort models, and effort and schedule estimates contribute to project planning and cost.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 139; Companion Slide 194. [Textbook Answer Key]

Q4.2. Which one is not a size measure for software?

(a) LOC · (b) Function Count · (c) Cyclomatic Complexity · (d) Halstead's program length

(c) Cyclomatic Complexity

Lines of Code (LOC), Function Count / Function-Point measures, and Halstead's Program Length ($N = N_1 + N_2$) are software-size measures. Function Count and the resulting Function Points are related steps, not interchangeable labels. Cyclomatic Complexity ($V(G) = e - n + 2P$), introduced by Thomas McCabe, is a structural complexity metric measuring the number of linearly independent paths through the control flow graph, not software size.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 143–150; Companion Slide 194. [Textbook Answer Key]

Q4.3. Function count method was developed by:

(a) B. Beizer · (b) B. Boehm · (c) M. Halstead · (d) Alan Albrecht

(d) Alan Albrecht

The Function Count / Function Point Analysis (FPA) method was originally developed by Allan J. Albrecht at IBM in 1979 to measure the functional size of software from the user's perspective, independent of programming language or implementation technology.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 145; Companion Slide 194. [Textbook Answer Key]

Q4.4. Function point analysis (FPA) method decomposes the system into functional units. The total number of functional units are:

(a) 2 · (b) 5 · (c) 4 · (d) 1

(b) 5

Function Point Analysis decomposes a software system into five distinct functional information domain characteristics: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 145–146; Companion Slide 194. [Textbook Answer Key]

Q4.5. IFPUG stands for:

(a) Initial function point uniform group · (b) International function point uniform group · (c) International function point user group · (d) Initial function point user group

> **(c) International function point user group**
>
> **IFPUG stands for the International Function Point Users Group.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 145; Companion Slide 195. [Textbook Answer Key]*

Q4.6. Function points can be calculated by:

(a) UFP * CAF · (b) UFP * FAC · (c) UFP * Cost · (d) UFP * Productivity

> **(a) UFP * CAF**
>
> **Final adjusted Function Points (FP) are calculated by multiplying Unadjusted Function Points (UFP) by the Complexity Adjustment Factor (CAF):** $\mathrm{FP} = \mathrm{UFP} \times \mathrm{CAF}$.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 147; Companion Slide 195. [Textbook Answer Key]*

Q4.7. Putnam resource allocation model is based on:

(a) Function points · (b) Norden/Rayleigh curve · (c) Putnam theory of software management · (d) Boehm's observation on manpower utilisation rate

> **(b) Norden/Rayleigh curve**
>
> **The Putnam Resource Allocation Model is founded directly upon Peter Norden's empirical observations at IBM, which established that manpower utilization throughout research and development projects follows a Rayleigh distribution curve over time.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 181–182; Companion Slide 195. [Textbook Answer Key]*

Q4.8. Manpower buildup for Putnam resource allocation model is:

(a) $D = K/t_d$ · (b) $D_0 = K/t_d^3$ · (c) $D = K/t_d^2$ · (d) $D_0 = K/t_d^2$

> **(b)** $D_0 = K/t_d^3$
>
> **In the Putnam resource allocation model, the difficulty metric is** $D = K/t_d^2$**, while the separate manpower build-up metric is** $D_0 = K/t_d^3$ **(persons/year^2).**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 186; Companion Slide 195. [Textbook Answer Key]*

Q4.9. COCOMO was developed initially by:

(a) B. W. Boehm · (b) Gregg Rothermel · (c) B. Beizer · (d) Rajiv Gupta

> **(a) B. W. Boehm**
>
> **The Constructive Cost Model (COCOMO '81) was developed by Barry W. Boehm.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 152; Companion Slide 196. [Textbook Answer Key]*

Q4.10. A COCOMO model is:

(a) Common Cost estimation model · (b) Constructive Cost Estimation model · (c) Complete cost estimation model · (d) Comprehensive Cost estimation model

> **(b) Constructive Cost Estimation model**
>
> **COCOMO is an acronym for COnstructive COst MOdel, an algorithmic cost estimation model that estimates software effort, schedule, and team size based on project size (KLOC) and cost drivers.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 152; Companion Slide 196. [Textbook Answer Key]*

Q4.11. Estimation of software development effort for organic software in COCOMO is:

(a) $E = 2.4(\text{KLOC})^{1.05}\ \text{PM}$ · (b) $E = 3.4(\text{KLOC})^{1.06}\ \text{PM}$ · (c) $E = 2.0(\text{KLOC})^{1.05}\ \text{PM}$ · (d) $E = 2.4(\text{KLOC})^{1.07}\ \text{PM}$

> **(a)** $E = 2.4(\text{KLOC})^{1.05}\ \text{PM}$
>
> **In Basic COCOMO '81, the nominal development effort equation for the Organic mode is** $E = 2.4 \times (\text{KLOC})^{1.05}\ \text{PM}$**.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 153; Companion Slide 196. [Textbook Answer Key]*

Q4.12. Estimation of size for a project is dependent on:

(a) Cost · (b) Schedule · (c) Time · (d) None of the above

> **(d) None of the above**
>
> **Software size is an intrinsic attribute of the system's requirements and functionality; it is independent of cost, schedule, or calendar time.** In fact, cost and schedule are dependent variables estimated *from* the estimated size, not the other way around.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 143; Companion Slide 196. [Textbook Answer Key]*

Q4.13. In function point analysis, number of Complexity Adjustment Factors are:

(a) 10 · (b) 20 · (c) 14 · (d) 12

> **(c) 14**
>
> **Function Point Analysis defines exactly 14 General System Characteristics (GSCs) as complexity adjustment factors (such as data communications, distributed data processing, performance, transaction rate, online data entry, etc.), each rated from 0 to 5.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 146–147; Companion Slide 196. [Textbook Answer Key]*

Q4.14. COCOMO-II estimation model is based on:

(a) Complex approach · (b) Algorithmic approach · (c) Bottom-up approach · (d) Top-down approach

(b) Algorithmic approach

COCOMO II is an algorithmic cost estimation model that uses mathematical formulas derived from empirical regression analysis of historical project metrics to predict effort and schedule.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 161; Companion Slide 197. [Textbook Answer Key]

Q4.15. Cost estimation for a project may include:

(a) Software Cost · (b) Hardware Cost · (c) Personnel Costs · (d) All of the above

(d) All of the above

The answer includes software cost, hardware cost, and personnel costs; these are all relevant components of a project cost estimate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 150; Companion Slide 197. [Textbook Answer Key]

Q4.16. In COCOMO model, if project size is typically 2–50 KLOC, then which mode is to be selected?

(a) Organic · (b) Semidetached · (c) Embedded · (d) None of the above

(a) Organic

In COCOMO, the Organic mode represents relatively small software projects (typically 2 to 50 KLOC) developed by experienced teams working in familiar, stable in-house environments with relaxed requirements.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 152; Companion Slide 197. [Textbook Answer Key]

Q4.17. COCOMO-II was developed at:

(a) University of Maryland · (b) University of Southern California · (c) IBM · (d) AT&T Bell Labs

(b) University of Southern California

COCOMO II was developed by Barry Boehm and his research team at the Center for Software Engineering (CSE) at the University of Southern California (USC) during the 1990s.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 161; Companion Slide 197. [Textbook Answer Key]

Q4.18. Which one is not a Category of COCOMO-II?

(a) End User Programming · (b) Infrastructure Sector · (c) Requirement Sector · (d) System Integration

(c) Requirement Sector

COCOMO II identifies four target project categories / application sectors: End-User Programming, Application Generators & Composition Aids, System Integration, and Infrastructure Sector.
"Requirement Sector" does not exist in COCOMO II taxonomy.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 162; Companion Slide 197. [Textbook Answer Key]

Q4.19. Which one is not an infrastructure software?

(a) Operating system · (b) Database management system · (c) Compilers · (d) Result management system

(d) Result management system

Infrastructure software provides core foundational services and platforms upon which application software runs; examples include Operating Systems, DBMS, Compilers, Network protocols, and Device Drivers. "Result management system" is an end-user application, not infrastructure software.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 162; Companion Slide 198. [Textbook Answer Key]

Q4.20. How many stages are in COCOMO-II?

(a) 2 · (b) 3 · (c) 4 · (d) 5

(b) 3

COCOMO II provides three increasingly detailed estimation stages tailored to the life cycle: Stage 1 (Application Composition Model), Stage 2 (Early Design Model), and Stage 3 (Post-Architecture Model).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 163; Companion Slide 198. [Textbook Answer Key]

Q4.21. Which one is not a stage of COCOMO-II?

(a) Application Composition estimation model · (b) Early design estimation model · (c) Post architecture estimation model · (d) Comprehensive cost estimation model

(d) Comprehensive cost estimation model

The three official stages of COCOMO II are Application Composition, Early Design, and Post-Architecture. "Comprehensive cost estimation model" is an invented distractor.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 163; Companion Slide 198. [Textbook Answer Key]

Q4.22. In Putnam resource allocation model, Rayleigh curve is modeled by the equation:

(a) $m(t) = 2at \cdot e^{-at^2}$ · (b) $m(t) = 2Kt \cdot e^{-at^2}$ · (c) $m(t) = 2Kat \cdot e^{-at^2}$ · (d) $m(t) = 2Kbt \cdot e^{-at^2}$

(c) $m(t) = 2Kat \cdot e^{-at^2}$

In the Putnam resource allocation model, manpower utilization rate $m(t)$ over time follows Norden's Rayleigh curve equation: $m(t) = 2Kat \cdot e^{-at^2}$, where K is total life-cycle effort, $a = \frac{1}{2t_d^2}$, and t is elapsed project time.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 182–183; Companion Slide 198. [Textbook Answer Key]

Q4.23. In Putnam resource allocation model, technology factor 'C' is defined as:

(a) $C = S \cdot K^{-1/3} \cdot t_d^{-4/3}$ · (b) $C = S \cdot K^{1/3} \cdot t_d^{4/3}$ · (c) $C = S \cdot K^{-1/3} \cdot t_d^{4/3}$ · (d) $C = S \cdot K^{1/3} \cdot t_d^{-4/3}$

(a) $C = S \cdot K^{-1/3} \cdot t_d^{-4/3}$

Putnam's Software Equation relates product size S to total effort K and development time t_d by $S = C \cdot K^{1/3} \cdot t_d^{4/3}$. Solving algebraically for the technology factor C yields $C = S \cdot K^{-1/3} \cdot t_d^{-4/3}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 187; Companion Slide 199. [Textbook Answer Key]

Q4.24. Risk management activities are divided in:

(a) 3 Categories · (b) 2 Categories · (c) 5 Categories · (d) 10 Categories

(b) 2 Categories

Risk management activities are divided into two fundamental primary categories: Risk Assessment (identification, analysis, prioritization) and Risk Control (planning, resolution, monitoring).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 200; Companion Slide 199. [Textbook Answer Key]

Q4.25. Which one is not a risk management activity?

(a) Risk assessment · (b) Risk control · (c) Risk generation · (d) None of the above

(c) Risk generation

The established activities of software risk management are Risk Assessment (evaluating exposure) and Risk Control (mitigating and monitoring risk). "Risk generation" is not one of the two listed management activities.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 200; Companion Slide 199. [Textbook Answer Key]

Q4.26. In the Putnam Resource Allocation Model, the peak manpower manning rate m_0 at time $t = t_d$ is mathematically given by:

(a) $m_0 = \frac{K}{t_d \cdot e}$ · (b) $m_0 = \frac{K}{t_d \sqrt{e}}$ · (c) $m_0 = \frac{2K}{t_d \sqrt{e}}$ · (d) $m_0 = \frac{K}{t_d^2 \sqrt{e}}$

(b) $m_0 = \frac{K}{t_d \sqrt{e}}$

Substituting $a = \frac{1}{2t_d^2}$ and $t = t_d$ into Norden's Rayleigh curve $m(t) = 2Kat \cdot e^{-at^2}$ yields $m_0 = 2K\left(\frac{1}{2t_d^2}\right) t_d \cdot e^{-\frac{t_d^2}{2t_d^2}} = \frac{K}{t_d} \cdot e^{-1/2} = \frac{K}{t_d \sqrt{e}}$. The expression with e alone in the denominator does not follow from this substitution; with $\sqrt{e} \approx 1.64872$, $m_0 \approx 0.6065 \frac{K}{t_d}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 184; Companion Slides 130 & 139. [Course Source Invariant]

Q4.27. In Putnam's Software Equation ($S = C_k \cdot K^{1/3} \cdot t_d^{4/3}$), compressing development time t_d by a factor of 2 while keeping product size S and technology C_k constant will cause required effort K to increase by a factor of:

(a) 2 · (b) 4 · (c) 8 · (d) 16

> **(d) 16**
>
> **Rearranging Putnam's software equation gives $K = \frac{S^3}{C_k^3 \cdot t_d^4}$, demonstrating that development effort K is inversely proportional to the fourth power of development time ($K \propto t_d^{-4}$).** If t_d is halved ($t_d/2$), effort increases by $(1/2)^{-4} = 2^4 = 16$ times.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 187–188; Companion Slide 166. [Course Source Invariant]*

Q4.28. In Intermediate COCOMO '81, how many total cost drivers (Effort Multipliers) are defined, and across how many categories are they grouped?

(a) 15 drivers across 4 categories · (b) 17 drivers across 4 categories · (c) 14 drivers across 3 categories · (d) 7 drivers across 2 categories

> **(a) 15 drivers across 4 categories**
>
> **Intermediate COCOMO defines exactly 15 cost drivers (effort multipliers) clustered into four primary categories: Product attributes (3: RELY, DATA, CPLX), Computer attributes (4: TIME, STOR, VIRT, TURN), Personnel attributes (5: ACAP, AEXP, PCAP, VEXP, LEXP), and Project attributes (3: MODP, TOOL, SCED).**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 155–158; Companion Slides 57–61. [Course Source Invariant]*

Q4.29. In COCOMO II, the effort exponent B for the Early Design and Post-Architecture models is computed from 5 scale factors (SF_j) using which formula?

(a) $B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j$ · (b) $B = 1.01 + 0.05 \sum_{j=1}^{5} SF_j$ · (c) $B = 0.91 + 0.1 \sum_{j=1}^{5} SF_j$ · (d) $B = 1.05 + 0.01 \sum_{j=1}^{5} SF_j$

> **(a)** $B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j$
>
> **In COCOMO II, the scale factor exponent B reflects economies or diseconomies of scale and is computed as $B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j$, where the five scale factors are Precedentedness (PREC), Development Flexibility (FLEX), Architecture/Risk Resolution (RESL), Team Cohesion (TEAM), and Process Maturity (PMAT).**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 169; Companion Slide 86. [Course Source Invariant]*

Q4.30. In Function Point Analysis, which information domain characteristic carries the highest possible individual complexity weight?

(a) External Inputs (EI) · (b) External Outputs (EO) · (c) Internal Logical Files (ILF) · (d) External Interface Files (EIF)

(c) Internal Logical Files (ILF)

In the function point weight matrix, Internal Logical Files (ILF) carry complexity weights of 7 (Low), 10 (Average), and 15 (High), which is the highest weight in the matrix. External Interface Files (EIF) carry 5, 7, 10; External Outputs (EO) carry 4, 5, 7; External Inputs (EI) carry 3, 4, 6.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 146; Companion Slide 16. [Course Source Invariant]

Q4.31. For the full-project Rayleigh curve, approximately what percentage of total effort K has accumulated from $t = 0$ to the release time $t = t_d$?

(a) 15% · (b) 25% · (c) 40% · (d) 65%

(c) 40%

**Integrating the full-project Rayleigh curve from $t = 0$ to $t = t_d$ gives cumulative effort $E(t_d) = K(1 - e^{-1/2}) = 0.3935K \approx 40\%K$. This cumulative value is distinct from the separately defined development-sub-cycle total $K_d = K/6$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 183; Companion Slide 151. [Course Source Invariant]

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- التكامل على منحنى المشروع من 0 إلى t_d يعطي جهداً تراكمياً يقارب 39.35% من K. هذا المقدار لا يساوي جهد دورة التطوير K_d؛ فهما كميتان مختلفتان ($K = 6K_d$).

Q4.32. Comparing single-variable static cost estimation models for a 100 KLOC project, which model predicts a higher nominal effort: Walston-Felix ($E = 5.2L^{0.91}$) or SEL ($E = 1.4L^{0.93}$)?

(a) Walston-Felix predicts significantly higher effort · (b) SEL predicts significantly higher effort · (c) Both predict identical effort · (d) Neither model can be applied at 100 KLOC

(a) Walston-Felix predicts significantly higher effort

For $L = 100$ KLOC: Walston-Felix yields $E = 5.2 \times (100)^{0.91} \approx 5.2 \times 66.07 \approx 343.6$ PM. SEL yields $E = 1.4 \times (100)^{0.93} \approx 1.4 \times 72.44 \approx 101.4$ PM. For these equations and $L = 100$ KLOC, Walston–Felix predicts more than three times the effort of SEL. This comparison alone does not identify the cause or establish which estimate is more accurate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 151; Companion Slide 37. [Course Source Invariant]

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- لهذا الحجم تعطي معادلة Felix–Walston جهداً أعلى من معادلة SEL؛ لا تحدد هذه المقارنة وحدها سبب الفرق أو أيهما أدق لمشروع معين.

Q4.33. In quantitative software risk analysis, Risk Exposure (RE) is defined mathematically as:

(a) $\mathrm{RE} = P + L$ · (b) $\mathrm{RE} = P \times L$ · (c) $\mathrm{RE} = L/P$ · (d) $\mathrm{RE} = P/L$

(b) $\mathrm{RE} = P \times L$

Risk Exposure (RE) is the product of the probability of incurring a loss due to a risk (P) and the potential magnitude of that loss (L): $\mathrm{RE} = P \times L$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 197; Companion Slide 191. [Course Source Invariant]

Q4.34. In COCOMO '81, if a project has high product complexity, strict hardware constraints, and an unfamiliar operational environment requiring rigorous interfaces, which mode best fits these conditions?

(a) Organic · (b) Semidetached · (c) Embedded · (d) Prototyping

(c) Embedded

Embedded mode best fits a project with tight hardware/software constraints and a rigid operational environment. Size ranges are typical guidance; the scenario's constraints also matter.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 152–153; Companion Slide 43. [Course Source Invariant]

Q4.35. What is the fundamental conceptual advantage of Function Points (FP) over Lines of Code (LOC) as a software size metric?

(a) FP can be counted automatically by a compiler · (b) FP is independent of the programming language used · (c) FP requires knowing the full source code syntax · (d) FP measures memory word allocation

(b) FP is independent of the programming language used

Function Points measure functionality delivered to the user from the external functional specification (SRS), making the metric completely programming language independent. In contrast, LOC is heavily dependent on programming language expressiveness, indentation styles, and compiler syntax, penalizing higher-level languages.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 147–148; Companion Slide 31. [Course Source Invariant]

CHAPTER 5 — SOFTWARE DESIGN

Multiple-Choice Questions (MCQ)

Section 1: Official Course-Slide MCQs (12 Items)

Q5.1. The most desirable form of coupling is:

(a) Control Coupling · (b) Data Coupling · (c) Common Coupling · (d) Content Coupling

> **(b) Data Coupling**
>
> **Modules communicate by passing data items as parameters.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 208; Companion Slides 16, 94.*

Q5.2. The worst type of coupling is:

(a) Content coupling · (b) Common coupling · (c) External coupling · (d) Data coupling

> **(a) Content coupling**
>
> **Content coupling is the tightest, most pathological, and worst form of coupling because one module directly modifies or branches into the internal procedural code, execution state, or local data of another module.** This destroys modular independence and prevents any isolated maintenance or unit testing.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 210; Companion Slide 24, 94.*

Q5.3. The most desirable form of cohesion is:

(a) Logical cohesion · (b) Procedural cohesion · (c) Functional cohesion · (d) Temporal cohesion

> **(c) Functional cohesion**
>
> **Functional cohesion represents the highest, strongest, and most desirable level of cohesion because all elements and instructions within the module cooperate to execute a single, well-defined functional task with no extraneous side effects.** Examples include mathematical functions like compute_square_root() or calculate_sine().
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 212; Companion Slide 30, 94.*

Q5.4. The worst type of cohesion is:

(a) Temporal cohesion · (b) Coincidental cohesion · (c) Logical cohesion · (d) Sequential cohesion

> **(b) Coincidental cohesion**
>
> **Coincidental cohesion is the weakest, least desirable, and lowest form of cohesion because instructions are bundled into a module arbitrarily with no meaningful functional, sequential, procedural, or temporal relationship.** It typically arises when random utility routines are lumped together without structural cohesion.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 26, 94.*

Q5.5. Which one is not a strategy for design?

(a) Bottom up design · (b) Top down design · (c) Embedded design · (d) Hybrid design

> **(c) Embedded design**
>
> **Software design methodology establishes three primary classical architectural strategies: Top-Down design, Bottom-Up design, and Hybrid (Mixed) design.** Embedded design is not an architectural decomposition strategy; in software engineering, "embedded" refers to an operational application category, hardware constraint, or a COCOMO development mode.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–215; Companion Slides 32–35, 94.*

Q5.6. Temporal cohesion means:

(a) Cohesion between temporary variables · (b) Cohesion between local variable · (c) Cohesion with respect to time · (d) Coincidental cohesion

> **(c) Cohesion with respect to time**
>
> **Temporal cohesion groups operations that are executed within the same window of time during program lifecycle execution, regardless of whether they share data or functional logic.** A canonical example is an application startup routine that initializes variables, opens log files, configures database handles, and draws UI frames simultaneously.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 28, 95.*

Q5.7. Functional cohesion means:

(a) Operations are part of single functional task and are placed in same procedures · (b) Operations are part of single functional task and are placed in multiple procedures · (c) Operations are part of multiple tasks · (d) None of the above

> **(a) Operations are part of single functional task and are placed in same procedures**
>
> **Functional cohesion occurs when all operations contained within a single procedure or module contribute directly and exclusively to executing one focused functional objective.** Breaking a single functional task across multiple arbitrary modules impairs cohesion and introduces undesirable stamp or control coupling.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 212; Companion Slide 30, 95.*

Q5.8. When two modules refer to the same global data area, they are related as:

(a) External coupled · (b) Data coupled · (c) Content coupled · (d) Common coupled

> **(d) Common coupled**
>
> **Common coupling occurs whenever two or more independent modules read, write, or bind to the same shared global data area (such as Fortran COMMON blocks, C global variables, or shared public memory pools).** If one module mutates the shared global format, all other modules referencing that pool risk runtime corruption.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 209; Companion Slide 22, 95.*

Q5.9. The module in which instructions are related through flow of control is:

(a) Temporal cohesion · (b) Logical cohesion · (c) Procedural cohesion · (d) Functional cohesion

> **(c) Procedural cohesion**
>
> **Procedural cohesion exists when elements inside a module are grouped together because they must be executed in a specific, predetermined sequence dictated by program control flow (such as step 1 followed by step 2 followed by step 3).** Unlike sequential cohesion, operations in procedural cohesion do not necessarily pass output data from one step to the next as input.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 27, 95.*

Q5.10. The relationship of data elements in a module is called:

(a) Coupling · (b) Cohesion · (c) Modularity · (d) None of the above

> **(b) Cohesion**
>
> **Cohesion measures the internal binding, intra-module strength, and semantic closeness of elements, statements, and data structures encapsulated within a single module.** Coupling, by contrast, measures inter-module relationship and external interdependence across different modules.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 210; Companion Slide 25, 96.*

Q5.11. A system that does not interact with external environment is called:

(a) Closed system · (b) Logical system · (c) Open system · (d) Hierarchal system

> **(a) Closed system**
>
> **In general systems theory applied to software architecture, a closed system is completely self-contained and operates in total isolation without accepting inputs from, or delivering outputs to, its external operating environment.** Real-world software systems are virtually always open systems because they interact continuously with users, sensors, and external networks.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 204; Companion Slide 6, 96.*

Q5.12. The extent to which different modules are dependent upon each other is called:

(a) Coupling · (b) Cohesion · (c) Modularity · (d) Stability

> **(a) Coupling**
>
> **Coupling is the qualitative and structural metric that gauges the degree of interdependence, interconnection, and mutual reliance between distinct software modules.** Good software engineering practice strives to minimize inter-module coupling while maximizing intra-module cohesion.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 207; Companion Slide 15, 96.*

Section 2: Curriculum Gap-Closing Synthesis MCQs (13 Items)

Q5.13. What is the fundamental operational distinction between Conceptual Design and Technical Design?

(a) Conceptual design specifies data structures and algorithms, while technical design gathers user stories.
(b) Conceptual design is written for the customer detailing *what* the system will do, while technical design is written for developers detailing *how* the system will be built.
(c) Conceptual design is performed after coding, while technical design precedes requirements elicitation.
(d) Conceptual design produces object code, while technical design produces test suites.

> **(b) Conceptual design is written for the customer detailing *what* the system will do, while technical design is written for developers detailing *how* the system will be built.**
>
> **Conceptual design establishes an external, user-oriented model that describes system capabilities and behavior from the stakeholder's perspective without exposing internal mechanics.** Technical design translates those conceptual abstractions into concrete implementation blueprints for software engineers, specifying internal data structures, module interfaces, algorithm choices, and system architecture.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 204–205; Companion Slides 4–5.*

Q5.14. Under Bertrand Meyer's five criteria for software modularity, which criterion requires that an error or exceptional condition occurring within a module remains confined to that module without crashing the entire system?

(a) Modular Decomposability · (b) Modular Composability · (c) Modular Continuity · (d) Modular Protection

> **(d) Modular Protection**
>
> **Modular Protection means that an abnormal condition occurring in a module should have its effects constrained within that module.**
>
> *Source: Bertrand Meyer,* Object-Oriented Software Construction*; Pressman (5th ed.), §13.4.3, printed p. 345.*

> **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - معيار الحماية النمطية (Modular Protection) عند Meyer ينص على حصر الأخطاء والاستثناءات داخل الوحدة ومنع انهيار النظام ككل. لا تخلطه مع Modular Continuity التي تعني حصر التغييرات في الكود عند تعديل المواصفات.

Q5.15. When two modules communicate by passing an entire composite data structure (such as a full Employee_Record struct), but the receiving module only uses a single field (Zip_Code), what type of coupling is created?

(a) Data Coupling · (b) Control Coupling · (c) Stamp Coupling · (d) External Coupling

> **(c) Stamp Coupling**
>
> **Stamp coupling occurs when an entire composite data structure or record layout is passed between modules when only a subset of its fields is required by the callee.** This exposes unnecessary fields to accidental manipulation and tightly couples both modules to the structural layout of the entire composite record.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 208; Companion Slide 17.*

Q5.16. In which form of cohesion do module elements operate on the exact same input data set to produce different, independent output values?

(a) Communicational Cohesion · (b) Sequential Cohesion · (c) Logical Cohesion · (d) Temporal Cohesion

> **(a) Communicational Cohesion**
>
> **Communicational cohesion occurs when all operations within a module manipulate or reference the identical input data structure or database record, but each operation produces distinct, separate results.** For example, a module that accepts a student record and concurrently computes GPA, evaluates credit standing, and formats a mailing address possesses communicational cohesion.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 29.*

Q5.17. Which design strategy begins by assembling and testing existing low-level reusable components and utility libraries, gradually constructing higher-level abstractions toward the root of the hierarchy?

(a) Top-Down Design · (b) Bottom-Up Design · (c) Stepwise Refinement · (d) Transform Centered Design

> **(b) Bottom-Up Design**
>
> **Bottom-up design starts with lower-level components and combines them into higher-level components.** It can be useful when reusable components already exist.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–214; Companion Slide 33.*

Q5.18. How does a Structure Chart fundamentally differ in semantic meaning from a standard Flowchart?

(a) A structure chart depicts runtime execution paths of algorithms, while a flowchart models class inheritance.
(b) A structure chart shows static calling hierarchy, module partitioning, and data/control couples, whereas a flowchart shows sequential instruction order and runtime control logic.
(c) Structure charts are used exclusively for object-oriented systems, while flowcharts are used for relational databases.
(d) There is no semantic difference; a structure chart is merely a modern graphic synonym for a flowchart.

(b) A structure chart shows static calling hierarchy, module partitioning, and data/control couples, whereas a flowchart shows sequential instruction order and runtime control logic.

A structure chart shows module hierarchy and the data or control information passed between modules. A flowchart shows the steps and branches of a procedure.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–218; Companion Slides 41–43.

Q5.19. In functional procedure layers, what is the primary role of the Structural Layer?

(a) Describing step-by-step algorithms using pseudocode.
(b) Specifying machine-executable byte code and hardware register assignments.
(c) Defining module boundaries, calling relationships, and inter-module interface contracts.
(d) Generating user-interface wireframes and customer surveys.

(c) Defining module boundaries, calling relationships, and inter-module interface contracts.

The structural layer of functional design notations models the macro-architecture of the software, establishing module decomposition, hierarchy, and inter-module coupling boundaries. It is complemented by the procedural layer, which articulates the internal algorithmic logic of each discrete module.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 220–221; Companion Slides 47–49.

Q5.20. Under IEEE Std 1016 (Recommended Practice for Software Design Descriptions), what is the core purpose of a Software Design Description (SDD)?

(a) To document developer employment contracts and billable hours.
(b) To serve as an authoritative technical blueprint communicating architecture, module interfaces, and design decisions among stakeholders.
(c) To replace compiler syntax checking with manual inspections.
(d) To act as a commercial marketing brochure for end users.

> **(b) To serve as an authoritative technical blueprint communicating architecture, module interfaces, and design decisions among stakeholders.**
>
> **IEEE Std 1016 specifies that the primary purpose of an SDD is to provide a precise, rigorous reference document that translates customer requirements into architectural blueprints, guiding coders during implementation, testers during verification, and maintainers during system evolution.**
>
> *Source: IEEE Std 1016-1998; K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 221–222; Companion Slides 50–52.*

Q5.21. Which statement best distinguishes an Abstract Data Type (ADT) from a class as presented in the supporting sources?

(a) An ADT specifies operations while hiding representation; a class groups objects with common structure and behavior. · (b) ADTs always use static binding, while classes always use dynamic binding. · (c) An ADT is a machine-level executable, while a class is only a user requirement. · (d) Classes cannot represent data abstractions.

> **(a) An ADT specifies operations while hiding representation; a class groups objects with common structure and behavior.**
>
> **An ADT is characterized by its operations rather than its internal representation. A class is a grouping of objects that share characteristics and behavior and can serve as their blueprint.** These descriptions are related but do not establish a universal class-versus-ADT capability checklist.
>
> *Source: Sommerville (9th ed.), §13.4, PDF p. 356; K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 227.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:** - لا تحفظ ادعاءات أن ADT لا يملك وراثة أو تعدد أشكال؛ هذا الفرق المحدد هو الذي تسنده الصفحات المذكورة.

Q5.22. In architectural structural design, what do the metrics "Fan-in" and "Fan-out" represent for a given module?

(a) Fan-in is the number of lines of code; Fan-out is the number of compiler errors.
(b) Fan-in is the number of modules that call the given module; Fan-out is the number of modules directly called by the given module.
(c) Fan-in is the number of input parameters; Fan-out is the return data type size.
(d) Fan-in is processor clock speed; Fan-out is network bandwidth consumption.

> **(b) Fan-in is the number of modules that call the given module; Fan-out is the number of modules directly called by the given module.**
>
> **High fan-in may indicate reuse. High fan-out may indicate that a module has many subordinate calls; inspect the design before deciding whether it needs restructuring.**
>
> *Source: Pressman (5th ed.), §13.4.5; Rajib Mall (4th ed.), §5.4, pp. 300–301.*

Q5.23. What fundamental architectural principle organizes modules around design decisions that each hides from other modules?

(a) Stepwise Refinement · (b) Information Hiding · (c) Temporal Coupling · (d) Data Pipelining

> **(b) Information Hiding**
>
> **Information hiding organizes modules around design decisions that are hidden from other modules.** A change to a hidden decision can then be localized when the interface remains suitable.
>
> *Source: David Parnas (1972); Pressman (5th ed.), §13.4.4, printed p. 352.*

Q5.24. During DFD-to-Structure-Chart mapping, what is the primary difference between Transform Analysis and Transaction Analysis?

(a) Transform analysis is used for real-time operating systems, while transaction analysis is used for compilers.
(b) Transform analysis identifies a central transform between afferent (input) and efferent (output) streams, while transaction analysis branches control from a central transaction center based on an action tag.
(c) Transform analysis converts classes into objects, while transaction analysis converts tables into SQL views.
(d) There is no structural difference; both algorithms generate identical single-branch structure charts.

> **(b) Transform analysis identifies a central transform between afferent (input) and efferent (output) streams, while transaction analysis branches control from a central transaction center based on an action tag.**
>
> **Transform analysis isolates the incoming data paths (afferent branch), the primary computational kernel (central transform), and outgoing result paths (efferent branch).** Transaction analysis applies when an input data item (transaction) contains an identifying type or opcode that triggers execution of one of several mutually exclusive operational action paths.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 218–220; Rajib Mall (4th ed.), §6.4.1.*

Q5.25. Can a non-trivial software system ever be completely decoupled, such that the degree of coupling between all modules is zero?

(a) Yes, complete decoupling is the mandatory standard requirement of object-oriented architectures.
(b) Yes, complete decoupling is achieved by eliminating all functions and writing all code in a single file.
(c) No, because modules must communicate data and transfer control to execute meaningful system tasks; a completely decoupled system would consist of isolated components unable to cooperate.
(d) No, because compiler standards mandate at least 15 global variables in every software product.

> **(c) No, because modules must communicate data and transfer control to execute meaningful system tasks; a completely decoupled system would consist of isolated components unable to cooperate.**
>
> **Complete decoupling ($Coupling = 0$) implies absolute independence with zero parameter passing, zero shared memory, zero message passing, and zero invocation.** Because software applications exist to solve complex problems through coordinated component collaboration, coupling cannot be entirely eliminated; software engineering aims to achieve *loose coupling* rather than *zero coupling*.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Exercise 5.24, printed p. 249; Companion Slide 15.*

Topic-to-Question Map

Topic Area	Questions Covering Item	Primary Course Sources	Key Concepts Covered
1. Software Design Fundamentals	Q5.11, Q5.13	K.K. Aggarwal pp. 203–206; Slides 2–9	Conceptual vs technical design, system boundary, closed systems, transition from requirements to architecture.
2. Modularity & Meyer's Criteria	Q5.14	Pressman §13.4.3; K.K. Aggarwal pp. 206–207	Modular decomposability, composability, understandability, continuity, protection.
3. Module Coupling Taxonomy	Q5.1, Q5.2, Q5.8, Q5.12, Q5.15, Q5.25	K.K. Aggarwal pp. 207–210; Slides 15–24	Data, stamp, control, external, common, content coupling; limits of complete decoupling.
4. Module Cohesion Taxonomy	Q5.3, Q5.4, Q5.6, Q5.7, Q5.9, Q5.10, Q5.16	K.K. Aggarwal pp. 210–213; Slides 25–30	Coincidental, logical, temporal, procedural, communicational, sequential, functional cohesion.
5. Design Strategies & Notations	Q5.5, Q5.17, Q5.18, Q5.19	K.K. Aggarwal pp. 213–221; Slides 32–49	Top-down, bottom-up, hybrid design; flowchart vs structure chart; functional procedure layers.
6. IEEE SDD (Std 1016-1998)	Q5.20	K.K. Aggarwal pp. 221–225; Slides 50–57	Scope, definitions, purpose, design entities, attributes, view representations.
7. Object-Oriented Design (OOD)	Q5.21, Q5.22	K.K. Aggarwal pp. 225–249; Slides 58–93	Objects, classes, ADT vs class, inheritance, polymorphism, encapsulation, reusable design guidelines.
8. Architectural & Structural Metrics	Q5.23, Q5.24	Pressman §13.4.5; Mall §5.4, §6.4.1	Fan-in/fan-out, information hiding, DFD transform analysis vs transaction analysis.

CHAPTER 6 — SOFTWARE METRICS

Multiple-Choice Questions (MCQ)

Section 1: Official Course-Slide MCQs (20 Items)

Q6.1. Which one is not a category of software metrics?

(a) Product metrics · (b) Process metrics · (c) Project metrics · (d) People metrics

(d) People metrics

In K.K. Aggarwal & Yogesh Singh's taxonomy (companion slides 9–10, 96; scan p. 254), software metrics are categorized into three primary classes:

1. **Product metrics: Measures of the software product at any development stage, ranging from requirements documents to source code and test suites** (e.g., size, complexity, reliability).
2. **Process metrics: Measures of the software development and maintenance process** (e.g., defect removal rate, development cycle duration, phase containment efficiency).
3. **Project metrics: Measures of project operational performance and resource utilization** (e.g., staffing allocation, cost tracking, schedule adherence, productivity).

"People metrics" is not one of the three established core categories in Aggarwal's classification.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 9–10, 96; scan p. 254.

Q6.2. Software science measures are developed by:

(a) M.Halstead · (b) B.Littlewood · (c) T.J.McCabe · (d) G.Rothermal

(a) M.Halstead

Software Science (also known as Halstead Metrics) was developed and published by Maurice H. Halstead in 1977. The theory operates on the premise that any computer program is a collection of tokens categorized strictly into operators and operands.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 255; companion slides 11, 96.

Q6.3. Vocabulary of a program is defined as:

(a) $\eta = \eta_1 + \eta_2$ · (b) $\eta = \eta_1 \times \eta_2$ · (c) $\eta = \eta_1 - \eta_2$ · (d) $\eta = \eta_1 / \eta_2$

(a) $\eta = \eta_1 + \eta_2$

The program vocabulary η represents the total number of distinct tokens appearing in a program. It is computed as the sum of:

- η_1**: Number of unique (distinct) operators.**
- η_2**: Number of unique (distinct) operands.**

Thus, $\eta = \eta_1 + \eta_2$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 256; companion slides 12, 96.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- ميز بدقة بين المفردات (Vocabulary η) وهو مجموع المعاملات والرموز الفريدة ($\eta_1 + \eta_2$)، وبين طول البرنامج (Length N) وهو مجموع التكرارات الكلية ($N_1 + N_2$).

Q6.4. In Halstead theory of software science, volume is measured in bits. The bits are:

(a) Number of bits required to store the program · (b) Actual size of a program if a uniform binary encoding scheme for vocabulary is used · (c) Number of bits required to execute the program · (d) None of the above

(b) Actual size of a program if a uniform binary encoding scheme for vocabulary is used

In Halstead's Software Science, Program Volume $V = N \log_2 \eta$ measures the information content or storage size of a program implementation.

Each of the N total tokens in the program requires a minimum binary representation of $\log_2 \eta$ bits (the minimum number of bits needed to distinguish among the η unique tokens in the program vocabulary under a uniform binary encoding). Therefore, the theoretical physical unit of Volume is **bits**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 14, 96.

Q6.5. In Halstead theory, effort is measured in:

(a) Person-months · (b) Hours · (c) Elementary mental discriminations · (d) None of the above

(c) Elementary mental discriminations

Halstead defined mental effort E as the total number of elementary mental discriminations (decisions) required to develop or comprehend a program. It is calculated as:

$$E = \frac{V}{L} = D \times V$$

Where D is the program difficulty ($D = 1/L = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$) and V is the volume in bits.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 259; companion slides 19, 97.

Q6.6. Language level is defined as:

(a) $\lambda = L^3 V$ · (b) $\lambda = LV$ · (c) $\lambda = LV^*$ · (d) $\lambda = L^2 V$

(d) $\lambda = L^2 V$

Language level λ is an invariant attribute of a programming language, representing the expressive power and conciseness of the language. It is derived from the relationship:

$$\lambda = L \times V^*$$

Since the program level is defined as $L = V^*/V$, substituting $V^* = L \times V$ yields:

$$\lambda = L \times (L \times V) = L^2 V$$

Higher-level programming languages possess higher values of λ.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 260; companion slides 23, 97.

Q6.7. Program weakness is:

(a) $WM = \overline{LV} \times \gamma$ · (b) $WM = \overline{LV} / \gamma$ · (a) $WM = \overline{LV} + \gamma$ · (d) None of the above

(d) None of the above [literal answer to the printed stem]

> [!WARNING] **Source-item defect:** The printed stem asks for **Program Weakness (WP)**, but choices (a)–(c) give formulas for **Module Weakness (WM)**; the source also duplicates the label `(a)` on its third formula choice. Since no choice gives the program-level formula, `(d)` None of the above is the literal answer to the stem as printed. If the intended stem was "Module weakness," the formula in (a) is the correct one: $WM = \overline{LV} \times \gamma$. Program weakness is the mean of module weaknesses: $WP = \frac{1}{m} \sum_{i=1}^{m} WM_i$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 278–280, 305; companion slides 68, 97.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- هذا السؤال يحمل خطأً مطبعياً ومفاهيمياً في أصل شريحة الناشر والكتاب (شريحة 97): سأل عن ضعف البرنامج (Program Weakness)، لكن جميع الخيارات تعرض صيغ ضعف الوحدة (WM).
- ضعف الوحدة هو: $WM = \overline{LV} \times \gamma$ حيث $\overline{LV}$ هو متوسط المتغيرات الحية لكل تعليمة تنفيذية، وγ هو متوسط عمر المتغيرات.
- ضعف البرنامج الإجمالي هو متوسط ضعف وحداته: $WP = \frac{1}{m} \sum WM_i$.

Q6.8. "FAN IN" of a component A is defined as:

(a) Count of the number of components that can call, or pass control, to component A · (b) Number of components related to component A · (c) Number of components dependent on component A · (d) None of the above

(a) Count of the number of components that can call, or pass control, to component A

Fan-in of a component A is the count of the number of components that can call, or pass control, to component A. It measures how many other components depend on component A.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 282; companion slides 69, 97.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- الـ Fan-in هو عدد الوحدات التي تستدعي الوحدة (تدفق داخل إليها). ارتفاعه دليل على إعادة استخدام الوحدة (High Reuse)، ولكنه يعني أيضاً حساسية عالية لأن أي عطل فيها يؤثر على كل من يستدعيها.

Q6.9. "FAN OUT" of a component A is defined as:

(a) Number of components related to component A · (b) Number of components dependent on component A · (c) Number of components that are called by component A · (d) None of the above

(c) Number of components that are called by component A

Fan-out of a component A is the number of components that are called by component A. It reflects the control dispersion and coordination responsibility of the component.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 282; companion slides 69, 98.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- الـ Fan-out هو عدد الوحدات الفرعية التي تستدعيها هذه الوحدة (تدفق خارج منها). ارتفاعه الشديد يشير إلى أن الوحدة تتحمل تعقيد تحكم وتنسيق مفرط وتحتاج إلى إعادة هيكلة (Refactoring).

Q6.10. Which is not a size metric?

(a) LOC · (b) Function count · (c) Program length · (d) Cyclomatic complexity

(d) Cyclomatic complexity

Cyclomatic Complexity is a control-flow structural complexity metric formulated by Thomas McCabe, designed to quantify procedural logic and the number of linearly independent paths. It is not a measure of software physical or functional size. In contrast, LOC, Function count, and Halstead's Program Length are recognized size metrics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 250, 422; companion slides 5, 98.

Q6.11. Which one is not a measure of software science theory?

(a) Vocabulary · (b) Volume · (c) Level · (d) Logic

(d) Logic

Halstead's Software Science suite includes Vocabulary (η), Length (N), Volume (V), Potential Volume (V^*), Program Level (L), Difficulty (D), Effort (E), Time (T), and Language Level (λ). There is no measure named "Logic" in Halstead's theory.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 255–262; companion slides 14–23, 98.

Q6.12. A human mind is capable of making how many number of elementary mental discriminations per second (i.e., Stroud number)?

(a) 5 to 20 · (b) 20 to 40 · (c) 1 to 10 · (d) 40 to 80

(a) 5 to 20

Halstead adopted psychologist John M. Stroud's perceptual research, which established that the human mind is capable of making between 5 and 20 elementary mental discriminations per second. In formulating practical Software Science equations, Halstead adopted the reference value $S = 18$ discriminations per second to convert mental effort E into estimated programming time in seconds:

$$T = \frac{E}{S} = \frac{E}{18} \text{ seconds}$$

Option (a) directly captures the empirical Stroud frequency range.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 260; companion slides 20, 98.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- ثابت ستروود (Stroud number) يحدد أن العقل البشري ينجز بين 5 و20 تمييزاً عقلياً في الثانية الواحدة. واعتمد هالستيد القيمة $S = 18$ في معادلات تحويل الجهد إلى زمن بالسكوند ($T = E/18$).

Q6.13. Minimal implementation of any algorithm was given the following name by Halstead:

(a) Volume · (b) Potential volume · (c) Effective volume · (d) None of the above

(b) Potential volume

Potential Volume V^* is the volume of the most compact, idealized version of the algorithm, typically conceived as a single call to a pre-existing function where only the unique input and output parameters are supplied. It is expressed as:

$$V^* = (2 + \eta_2^*) \log_2 (2 + \eta_2^*)$$

Where η_2^* represents the number of conceptually essential input/output operands.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 15, 99.

Q6.14. Program volume of a software product is:

(a) $V = N \log_2 \eta$ · (b) $V = (N/2) \log_2 \eta$ · (c) $V = 2N \log_2 \eta$ · (d) $V = N \log_2 \eta + 1$

(a) $V = N \log_2 \eta$

Program volume is computed as total program length N multiplied by the binary logarithm of the vocabulary η:

$$V = N \log_2 \eta = (N_1 + N_2) \log_2 (\eta_1 + \eta_2)$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 14, 99.

Q6.15. Which one is the international standard for size measure?

(a) LOC · (b) Function count · (c) Program length · (d) None of the above

No unique key can be verified from the supplied course item. The stem says "international standard for size measure," while the options are LOC, Function count, Program length, and None of the above. The source does not define what "international standard" means here or provide an answer key. Choice (b) may be the intended classroom answer if "Function count" is being used loosely for function-oriented sizing, but that interpretation is not established by the item itself.

Source item: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slide 99; scan p. 305. No publisher key located.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- السؤال وخياراته لا تكفي لإثبات مفتاح وحيد؛ احتمال (b) تفسير للنية فقط، وليس مفتاحاً مؤكداً.

Q6.16. Which one is not an object oriented metric?

(a) RFC · (b) CBO · (c) DAC · (d) OBC

(d) OBC

Response For a Class (RFC) and Coupling Between Objects (CBO) belong to the seminal Chidamber & Kemerer (CK) suite, and Data Access Component (DAC) belongs to the Li & Henry object-oriented metrics suite.

"OBC" is not a recognized metric in software engineering literature.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–293; companion slides 80–84, 99.

Q6.17. Which metric also consider indirect connected methods?

(a) TCC · (b) LCC · (c) Both of the above · (d) None of the above

(b) LCC

Bieman & Kang (1995) defined two cohesion metrics based on attribute sharing among methods:

- **Tight Class Cohesion (TCC): Measures the relative number of directly connected method pairs** (two methods are directly connected if they access at least one common instance variable).
- **Loose Class Cohesion (LCC): Measures the relative number of directly or indirectly connected method pairs** (two methods are indirectly connected if they are connected through a sequence of intermediate method accesses).

Therefore, $LCC \geq TCC$ always holds.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 292–293; companion slides 85, 100.

Q6.18. Depth of inheritance tree (DIT) can be measured by:

(a) Number of ancestors classes · (b) Number of successor classes · (c) Number of failure classes · (d) Number of root classes

(a) Number of ancestors classes

In companion slide 100 and textbook scan p. 305, Depth of Inheritance Tree (DIT) is defined as the number of ancestor classes (the maximum path length from the class node to the root class of the inheritance tree). Successor classes correspond to descendants/subclasses (measured by NOC - Number of Children).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 289; companion slides 81, 100.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- يقاس عمق شجرة الوراثة (DIT) بعدد الكلاسات السلفية (Number of ancestor classes) صعوداً إلى كلاس الجذر.
- في الوراثة المتعددة (Multiple Inheritance)، يؤخذ المسار الأطول إلى الجذر.

Q6.19. A dynamic page is:

(a) Where contents are not dependent on the actions of the user · (b) Where contents are dependent on the actions of the user · (c) Where contents cannot be displayed · (d) None of the above

> **(b) Where contents are dependent on the actions of the user**
>
> **Dynamic web pages are constructed on demand on the web server by executing server-side scripts and querying backend databases, where contents are dependent on the actions, requests, or session state of the user.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 91, 100.*

Q6.20. Which of the following is not a size metric?

(a) LOC · (b) FP · (c) Cyclomatic Complexity · (d) Program length

> **(c) Cyclomatic Complexity**
>
> **Cyclomatic Complexity is a control-flow structural complexity metric measuring linearly independent basis paths, not an attribute of software size.** In contrast, LOC, FP, and Halstead's Program Length are recognized physical or functional size metrics.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 5, 98, 100.*

Section 2: Syllabus Gap-Closing Synthesis MCQs (15 Items)

Q6.21. In measurement theory (Stevens' typology), which measurement scale possesses a non-arbitrary, true absolute zero point?

(a) Nominal scale · (b) Ordinal scale · (c) Interval scale · (d) Ratio scale

> **(d) Ratio scale**
>
> **A Ratio scale has ordered values, equal intervals, and a non-arbitrary zero. Ratios are meaningful on this scale; for example, 2,000 LOC is twice 1,000 LOC when both counts use the same rules.** An Interval scale has equal intervals but an arbitrary zero.
>
> *Source: Fenton, "Software Measurement: A Necessary Scientific Basis," pp. 200–201, https://doi.org/10.1109/32.268921; Aggarwal & Singh (3rd ed.), scan p. 252.*

Q6.22. Which of the following metrics is classified as a direct measure of an internal software attribute?

(a) System Reliability (Mean Time Between Failures) · (b) Developer Productivity (LOC per Person-Month) · (c) Source Code Length (Lines of Code) · (d) System Maintainability

(c) Source Code Length (Lines of Code)

Lines of Code (LOC) is a direct measure because it can be assessed strictly by examining the entity itself without calculating or observing other attributes. Furthermore, it measures an **internal attribute** because it depends solely on the source code artifact itself, independent of runtime behavior or the external operating environment. MTBF and Maintainability are external attributes; Productivity is an indirect process metric.

Source: Sommerville, I., "Software Engineering", 9th ed., printed pp. 668–671.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- المقياس المباشر (Direct Measure) يُقاس مباشرة من الكيان دون الحاجة لمعادلة تشتق قيمته من متغيرات أخرى.
- السمة الداخلية (Internal Attribute) تخص الكيان بمفرده (مثل حجم الكود LOC أو تعقيده)، بينما السمة الخارجية (External Attribute) تظهر فقط أثناء التشغيل والاستخدام (مثل الاعتمادية وسهولة الصيانة).

Q6.23. In the Goal–Question–Metric (GQM) paradigm, what is the correct hierarchical order from top to bottom?

(a) Conceptual Level → Operational Level → Quantitative Level · (b) Operational Level → Quantitative Level → Conceptual Level · (c) Quantitative Level → Conceptual Level → Operational Level · (d) Metric → Goal → Question

(a) Conceptual Level → Operational Level → Quantitative Level

The Goal–Question–Metric (GQM) framework is structured across three formal hierarchical levels:

1. **Conceptual Level (Goal): A high-level business or engineering objective is established for an object with respect to various quality models from a specific viewpoint.**
2. **Operational Level (Question): A set of specific questions is formulated to characterize the assessment or achievement of each goal.**
3. **Quantitative Level (Metric): A set of objective or subjective metrics is associated with each question to answer it quantitatively.**

Source: Leach, R., printed pp. 39–40.

Q6.24. According to Sommerville, what is the fundamental difference between predictor metrics and control metrics?

(a) Predictor metrics measure hardware; control metrics measure software · (b) Predictor metrics forecast future attributes; control metrics monitor active process progress · (c) Predictor metrics are subjective; control metrics are automated · (d) Predictor metrics apply only to Agile; control metrics apply to Waterfall

(b) Predictor metrics forecast future attributes; control metrics monitor active process progress

Software metrics fulfill two distinct management functions:

- **Predictor metrics: Measure current internal attributes to forecast future internal or external attributes of software** (e.g., using cyclomatic complexity and size to predict future defect density or maintenance cost).
- **Control metrics: Provide real-time visibility into an active process to allow managers to control development progress against plans** (e.g., tracking average days to resolve defect reports or schedule burn-down rate).

Source: Sommerville, I., 9th ed., printed pp. 668–670.

Q6.25. Which of the following is considered a primary technical criticism of Lines of Code (LOC) as a productivity metric?

(a) It cannot be counted automatically by tools · (b) It penalizes expressive, high-level languages and rewards verbose code · (c) It is not accepted by the IEEE · (d) It cannot be measured in procedural languages

(b) It penalizes expressive, high-level languages and rewards verbose code

Because LOC measures physical syntax rather than functional utility, developers writing in concise, high-level languages (e.g., Python, SQL) produce far fewer lines of code to achieve the exact same functionality as a verbose implementation in C or Assembly. Calculating productivity as LOC/PM creates a false paradox where verbose, bloated implementations appear to have higher productivity than clean, expressive solutions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–142; Pressman, 5th ed., printed pp. 92–95.

Q6.26. In Albrecht's Function Point Analysis, which of the following is an external functional component?

(a) Internal Logical Files (ILF) · (b) External Inquiries (EQ) · (c) General System Characteristics (GSC) · (d) Value Adjustment Factor (VAF)

(b) External Inquiries (EQ)

Albrecht's Function Point model identifies five primary information domain components:

1. **External Inputs (EI):** Elementary processes that process data entering from outside.
2. **External Outputs (EO):** Elementary processes that send derived data outside.
3. **External Inquiries (EQ):** Elementary input-output combinations that retrieve data immediately without updating files.
4. **Internal Logical Files (ILF):** User-identifiable groups of logically related data maintained entirely inside the application boundary.
5. **External Interface Files (EIF):** Files maintained by other systems referenced by the application.

EQ is an external transaction component. ILF is internal; GSC and VAF are adjustment factors, not information domain components.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 142–146; companion slides Ch.4.

Q6.27. What is the standard range of the Value Adjustment Factor (VAF) in Albrecht's Function Point calculation?

(a) 0.00 to 1.00 · (b) 0.65 to 1.35 · (c) 0.80 to 1.20 · (d) 1.00 to 2.00

(b) 0.65 to 1.35

The Value Adjustment Factor (VAF) is derived from 14 General System Characteristics (F_1 to F_{14}), each rated from 0 (no influence) to 5 (strong influence):

$$VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$$

- **Minimum possible VAF:** When all $F_i = 0$, $VAF = 0.65 + 0.01(0) = 0.65$ ($\pm 35\%$ adjustment downwards).
- **Maximum possible VAF:** When all $F_i = 5$, $\sum F_i = 70$, $VAF = 0.65 + 0.01(70) = 1.35$ ($\pm 35\%$ adjustment upwards).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 145; Ch.4 companion slides.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- يتراوح معامل تعديل القيمة (VAF) دوماً بين 0.65 و1.35، وهو ما يعطي تعديلاً أقصاه $\pm 35\%$ على النقاط الوظيفية غير المعدلة (UFP).

Q6.28. In Object Points sizing (as used in COCOMO II Early Prototyping), how are software objects identified?

(a) Number of C++ class definitions · (b) Number of screens, reports, and 3GL components · (c) Number of database tables and foreign keys · (d) Number of use cases and actors

(b) Number of screens, reports, and 3GL components

Object Points (also known as Application Points) is a high-level functional size metric computed during early prototyping before detailed code specifications exist. System size is quantified by counting screens, reports, and third-generation language (3GL) modules, classifying each as simple, medium, or difficult.

Source: Mall, R., "Fundamentals of Software Engineering", 4th ed., printed pp. 165–166.

Q6.29. In Halstead's Software Science, what is the mathematical relationship between Program Difficulty D and Program Level L?

(a) $D = L$ · (b) $D = 1/L$ · (c) $D = L^2$ · (d) $D = \log_2 L$

(b) $D = 1/L$

Program difficulty D is defined as the reciprocal of the program level L:

$$D = \frac{1}{L} = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$$

As the program level decreases (indicating greater implementation verbosity or complexity), the difficulty of comprehending or writing the program increases proportionally.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 259; companion slides 18–19.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- صعوبة البرنامج (D) هي المعكوس الدقيق لمستوى البرنامج (L): $D = 1/L$. صيغة حسابها العملية هي: $(\eta_1/2) \times (N_2/\eta_2)$.

Q6.30. In Henry & Kafura's Information Flow metric (as documented in Pressman 5e §19.4.2), what is the formula for the complexity of module i?

(a) $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]$ · (b) $HKM = \text{length}(i) \times [fan_in(i) \times fan_out(i)]^2$ · (c) $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$ · (d) $HKM = \frac{fan_in(i) + fan_out(i)}{\text{length}(i)}$

(c) $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$

As verified in Pressman 5th ed. §19.4.2 (printed p. 524 / PDF p. 552), the Henry and Kafura Metric (HKM) computes module structural complexity as:

$$HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$$

Where $\text{length}(i)$ is the module size in lines of code, $fan_in(i)$ includes control connections and data structures retrieved, and $fan_out(i)$ includes modules called and data structures updated. The squared term represents the non-linear potential communication paths.

Source: Pressman, R., "Software Engineering: A Practitioner's Approach", 5th ed., printed p. 524.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- معادلة هنري وكافورا المعتمدة في مرجع Pressman §19.4.2 هي: الحجم مضروب في مربع مجموع (fan-in + fan-out). التربيع يمثل النمو غير الخطي لقنوات الاتصال المحتملة.

Q6.31. For a strongly connected control flow graph with E edges, N nodes, and P connected components, McCabe's Cyclomatic Complexity $V(G)$ is:

(a) $V(G) = E - N + P$ · (b) $V(G) = E - N + 2P$ · (c) $V(G) = N - E + 2P$ · (d) $V(G) = E + N - P$

(b) $V(G) = E - N + 2P$

Thomas McCabe defined cyclomatic complexity using graph theory for a control flow graph G as:

$$V(G) = E - N + 2P$$

For a single single-entry, single-exit module where $P = 1$, the formula reduces to $V(G) = E - N + 2$. It can also be computed as the number of predicate (decision) nodes plus one ($P_{pred} + 1$), or the number of bounded enclosed regions plus one.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–423; companion slides Ch.8.

Q6.32. In the Chidamber & Kemerer metric suite, Lack of Cohesion in Methods (LCOM) quantifies:

(a) The percentage of lines that contain comments · (b) The number of method pairs that do not share instance variables minus the number that do · (c) The depth of inheritance · (d) The number of parameters per method

(b) The number of method pairs that do not share instance variables minus the number that do

LCOM measures the dissimilarity of methods in a class based on shared attribute access. Let P be the number of method pairs that share no instance variables, and Q be the number of method pairs that share at least one instance variable. Then:

$$LCOM = \begin{cases} P - Q & \text{if } P > Q \\ 0 & \text{otherwise} \end{cases}$$

A high LCOM indicates low cohesion, suggesting that the class performs disparate tasks and should be refactored into smaller subclasses.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 292–293; companion slides 84–85.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- مقياس LCOM يقيس ضعف أو غياب التماسك، فكلما كبرت قيمته دل ذلك على سوء التصميم وتشتت الكلاس، وإذا كان عدد الأزواج المشتركة أكبر من غير المشتركة يُسند الصفر ($LCOM = 0$).

Q6.33. Which expression gives the unadjusted Use Case Point total using the course actor and use-case weights?

(a) $UUCP = UAW + UUCW$ · (b) $UUCP = UAW \times UUCW$ · (c) $UUCP = TCF + ECF$ · (d) $UUCP = LOC/PM$

(a) $UUCP = UAW + UUCW$

The unadjusted total is the sum of the weighted actor count (UAW) and weighted use-case count ($UUCW$). The inspected course material does not establish additional adjustment factors or an effort conversion.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 295–296; companion slides 89–90.

Q6.34. Defect Removal Efficiency (DRE) is mathematically defined as:

(a) $DRE = E/(E + D)$ · (b) $DRE = D/(E + D)$ · (c) $DRE = (E + D)/E$ · (d) $DRE = E \times D$

(a) $DRE = E/(E + D)$

Defect Removal Efficiency (DRE) measures the quality filtering ability of software review and testing activities. It is defined as:

$$DRE = \frac{E}{E + D}$$

- E**: Number of defects discovered during the software development phase (before release).**
- D**: Number of defects discovered by customers or downstream testing phases (after release).**

A DRE approaching 1.0 (or 100%) indicates that virtually all defects were caught and eradicated internally prior to delivery.

Source: Pressman, R., 5th ed., printed p. 98 / PDF p. 126; Aggarwal & Singh, companion slide 10.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- كفاءة إزالة العيوب (DRE) هي نسبة الأخطاء المكتشفة داخلياً قبل التسليم (E) إلى إجمالي الأخطاء ($E + D$). كلما اقتربت من 1.0 كانت اختبارات ومراجعات الفريق مثالية.

Q6.35. Which organizational pitfall describes developers deliberately changing their work habits to inflate a tracked metric at the expense of true software quality?

(a) Brooks' Law · (b) Measurement Dysfunction / Gaming · (c) Pareto Principle · (d) Software Entropy

(b) Measurement Dysfunction / Gaming

Metric gaming is a possible form of measurement dysfunction: when a narrow measure becomes an incentive target, people may optimize the reported number instead of the intended outcome. For example, a LOC bonus could encourage unnecessary code, while a closed-ticket target could encourage prioritizing easy tickets. These are possible scenario risks, not guaranteed effects or claims attributed to a named law.

Source: Aggarwal & Singh, companion slides 7–8, for implementation problems; scenario examples are authored.

Verify distinguishing criteria between Halstead, McCabe, Information Flow, and Object-Oriented metric families.

ASE CHAPTER 7

T1 Multiple-Choice Questions Pack

60 MCQs

Section 1: Official Course-Slide MCQs (50 Items)

Q7.1. Which one is not a phase of "bath tub curve" of hardware reliability?

(a) Burn-in · (b) Useful life · (c) Wear-out · (d) Test-out

(d) Test-out

The classical hardware reliability "bathtub curve" depicts failure rate over time across three distinct operational phases:

1. **Burn-in phase (Infant mortality): Characterized by an initially high failure rate that decreases rapidly as manufacturing defects and marginal components are identified and replaced.**
2. **Useful life phase: Characterized by a low, relatively constant failure rate driven by random, unpredictable environmental stresses.**
3. **Wear-out phase: Characterized by a steadily increasing failure rate caused by physical deterioration, fatigue, oxidation, and mechanical friction.**

"Test-out" is not a recognized operational phase of the bathtub curve.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 308; companion slides 3, 108.

📝 **Academic Note**

ملاحظة الطالب والفخ الامتحاني:

- المراحل الثلاث لمنحنى الحوض (Bathtub curve) في عتاد الأجهزة هي: مرحلة البداية (Burn-in)، ثم العمر النافع المستقر (Useful life)، ثم مرحلة الاهتراء والتقادم (Wear-out).
- لا توجد مرحلة باسم «Test-out»، فالبرمجيات لا تهترئ مادياً (No wear-out) بعكس الهاردوير.

Q7.2. Software reliability is:

(a) The probability of failure free operation of a program for a specified time in a specified environment · (b) The probability of failure of a program for a specified time in a specified environment · (c) The probability of success of a program for a specified time in any environment · (d) None of the above

(a) The probability of failure free operation of a program for a specified time in a specified environment

Software reliability is the probability of failure-free operation of a program for a specified time in a specified environment.

The definition specifies four dimensions:

- **Probability:** A mathematical quantification bounded in $[0, 1]$.
- **Failure-free operation:** The absence of dynamic non-conformances with user expectations.
- **Specified time:** Measured in execution (CPU) time, clock time, or operational units.
- **Specified environment:** The input space and operational profile under which the software executes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 308–309; companion slides 6, 108.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- احفظ أركان تعريف اعتمادية البرمجيات الأربعة: الاحتمالية (Probability) ← التشغيل الخالي من الفشل (Failure-free) ← في زمن محدد (Specified time) ← في بيئة تشغيلية محددة (Specified environment).

Q7.3. Fault is:

(a) Defect in the program · (b) Mistake in the program · (c) Error in the program · (d) All of the above

(d) All of the above

In IEEE standard terminology and software reliability foundations, a fault is an internal manifestation of a human mistake or error that introduces a defect into the program source code, design, or documentation. While rigorous taxonomies distinguish the developer's cognitive error (mistake) from the static code defect (fault) and the runtime event (failure), the terms defect, mistake, and error are commonly used interchangeably in introductory reliability literature to characterize a fault.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 310; companion slides 8, 108.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تسلسل الأعطال القياسي: الخطأ البشري (Error/Mistake) يُنتج عيباً داخلياً في الكود يسمى العلة أو الخلل (Fault/Defect)، والذي يسبب عند تنفيذه الفشل التشغيلي الخارجي (Failure).

Q7.4. One fault may lead to:

(a) One failure · (b) Two failures · (c) Many failures · (d) All of the above

(d) All of the above

There is no strict one-to-one mapping between static faults and dynamic failures:

- **A single fault may manifest as a single failure** under a specific test case.
- **A single fault may trigger multiple recurring failures** across different user transactions or continuous iterations.
- **A single fault may cause an entire cascade of failures** in downstream distributed components.
- Conversely, **multiple distinct faults may combine to trigger a single observable failure.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 310; companion slides 8, 108.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- العلاقة بين العيب المصدري (Fault) والفشل التشغيلي (Failure) هي علاقة (One-to-Many)؛ عيب برمجي واحد في حلقة تكرار قد يسبب آلاف الانهيارات، وقد لا يظهر أبداً إذا لم يُنفذ مساره.

Q7.5. Which "time" unit is not used in reliability studies?

(a) Execution time · (b) Machine time · (c) Clock time · (d) Calendar time

(b) Machine time

Software reliability engineering characterizes system behavior across three specific, well-defined time concepts:

1. **Execution time (τ): The actual processor (CPU) time spent by the computer system executing the instructions of the software under study.**
2. **Calendar time (t): The ordinary chronological time experienced in calendar days, weeks, or months**, including idle time and off-shift hours.
3. **Clock time: The total elapsed wall-clock running time during which the system is operational.**

"Machine time" is an ambiguous, non-standard term not formally recognized as a time dimension in reliability modeling.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 310–311; companion slides 11, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- المقاييس الزمنية المعتمدة في دراسات الاعتمادية هي ثلاثة: زمن التنفيذ الفعلي للمعالج (τ Execution time)، والزمن التقويمي الطبيعي (t Calendar time)، وزمن الساعة الجدارية (Clock time). مصطلح «Machine time» مستبعد.

Q7.6. Failure occurrences can be represented as:

(a) Time to failure · (b) Time interval between failures · (c) Failures experienced in a time interval · (d) All of the above

(d) All of the above

In stochastic reliability modeling, the random failure process can be mathematically formulated in three mathematically equivalent representations:

1. **Time to failure:** The elapsed time from the start of execution until failure occurrence.
2. **Time interval between failures:** The incremental operational duration separating successive failure events ($t_i - t_{i-1}$).
3. **Failures experienced in a time interval:** The cumulative or incremental count of failures observed over a given operational duration ($M(\tau)$ or $\Delta\mu$).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 311; companion slides 12, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- يمكن تمثيل حدوث الانهيارات بثلاث طرق إحصائية متكافئة: زمن حدوث الفشل، أو الفاصل الزمني بين فشلين، أو عدد الإخفاقات خلال فترة محددة.

Q7.7. Maximum possible value of reliability is:

(a) 100 · (b) 10 · (c) 1 · (d) 0

(c) 1

Because reliability $R(t)$ is mathematically defined as a probability, its values are strictly bounded by the probability axioms within the closed real interval:

$$0 \leq R(t) \leq 1$$

A reliability value of $R(t) = 1.0$ represents a 100% mathematical probability of failure-free operation over the specified interval. (While percentages such as 100% are colloquial, the raw mathematical maximum is 1).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 309; companion slides 7, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الاعتمادية هي دالة احتمالية رياضية (Probability)، وبالتالي فإن قيمتها الصغرى هي 0 وقيمتها العظمى هي 1 (ولا يُختار 100 إلا إذا وُضعت علامة النسبة المئوية %).

Q7.8. Minimum possible value of reliability is:

(a) 100 · (b) 10 · (c) 1 · (d) 0

(d) 0

The lower bound of the probability distribution for software reliability is 0, representing complete operational failure where failure-free operation over the specified duration cannot occur.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 309; companion slides 7, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- القيمة الصغرى المطلقة للاعتمادية هي 0.

Q7.9. As the reliability increases, failure intensity:

(a) Decreases · (b) Increases · (c) No effect · (d) None of the above

(a) Decreases

Reliability $R(t)$ and failure intensity $\lambda(t)$ share an inverse exponential relationship:

$$R(t) = \exp\left(-\int_0^t \lambda(u)\,du\right)$$

When failure intensity λ decreases (indicating fewer failures occurring per unit of execution time), the probability of failure-free operation $R(t)$ increases toward 1. Conversely, high failure intensity rapidly drives reliability toward zero.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 312, 321; companion slides 21, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- العلاقة عكسية صريحة: كلما انخفضت شدة الفشل (Failure Intensity λ)، ارتفعت موثوقية واعتمادية النظام (R).

Q7.10. If failure intensity is 0.005 failures/hour during 10 hours of operation of a software, its reliability can be expressed as:

(a) 0.10 · (b) 0.92 · (c) 0.95 · (d) 0.98

(c) 0.95

Under the constant failure intensity assumption over operational duration t, reliability is calculated using the exponential reliability formula:

$$R(t) = e^{-\lambda t}$$

Substituting $\lambda = 0.005$ failures/hour and $t = 10$ hours:

$$R(10) = e^{-(0.005 \times 10)} = e^{-0.05} \approx 0.951229 \approx 0.95$$

Therefore, the probability of failure-free operation during the 10-hour interval is approximately 0.95 (or 95.1%).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 312; companion slides 22, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- القانون الرياضي المباشر: $R = e^{-\lambda t} = e^{-(0.005 \times 10)} = e^{-0.05} \approx 0.9512$. التقريب الصحيح هو 0.95.

Q7.11. Software Quality is:

(a) Conformance to requirements · (b) Fitness for the purpose · (c) Level of satisfaction · (d) All of the above

(d) All of the above

Software quality is a multidimensional concept synthesizing several canonical engineering perspectives:

- **Crosby's definition:** "Conformance to requirements" (meeting specified functional and non-functional specifications).
- **Juran's definition:** "Fitness for purpose / use" (satisfying customer operational needs in real production).
- **IEEE / ISO definition:** The degree to which a software system satisfies user requirements and produces a high level of stakeholder satisfaction.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 321; companion slides 24, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- جودة البرمجيات تشمل التوافق مع المتطلبات المحددة (Crosby)، والملاءمة للغرض التشغيلي (Juran)، وتحقيق رضا المستخدم (ISO/IEEE).

Q7.12. Defect rate is:

(a) Number of defects per million lines of source code · (b) Number of defects per function point · (c) Number of defects per unit of size of software · (d) All of the above

(d) All of the above

Defect rate (or defect density) is defined as the total number of identified defects normalized against a standardized measure of software size. Industry and academic standards compute defect rate per KLOC (thousands of lines of code), per million lines of code (in telecommunications/avionics), or per Function Point (in business IT systems). Therefore, any metric normalizing defects per unit of size represents defect rate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 322; companion slides 27, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- معدل أو كثافة العيوب (Defect rate) هو مقياس منسوب لوحدة الحجم، سواء قيس بعدد العيوب لكل KLOC أو لكل Function Point.

Q7.13. How many product quality factors have been proposed in McCall quality model?

(a) 2 · (b) 3 · (c) 11 · (d) 6

(c) 11

In Jim McCall's seminal software quality model (1977), software quality is decomposed into exactly 11 product quality factors organized across three operational perspectives:

1. **Product Operation (5 factors):** Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision (3 factors):** Maintainability, Flexibility, Testability.
3. **Product Transition (3 factors):** Portability, Reusability, Interoperability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 323–324; companion slides 30–35, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- نموذج ماكول (McCall) يقسم الجودة إلى **3 أبعاد رئيسية** تضم بداخلها **11 عاملاً نوعياً (Factors)**: 5 للتشغيل + 3 للمراجعة والتعديل + 3 للانتقال وقابلية النقل.

Q7.14. Which one is not a product quality factor category of McCall quality model?

(a) Product revision · (b) Product operation · (c) Product specification · (d) Product transition

(c) Product specification

McCall's triangular quality framework categorizes its 11 factors into three broad operational categories:

- **Product Operation:** Using the software to perform real-world tasks.
- **Product Revision:** Modifying and debugging the software code.
- **Product Transition:** Porting and adapting the software to new platforms.

"Product specification" is an engineering phase, not one of McCall's three quality categories.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 30, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- أبعاد ماكول الثلاثة الشهيرة هي: Product Operation وProduct Revision وProduct Transition. لا يوجد بعد باسم «Product specification».

Q7.15. The second level of quality attributes in McCall quality model are termed as:

(a) Quality criteria · (b) Quality factors · (c) Quality guidelines · (d) Quality specifications

(a) Quality criteria

McCall's quality model is structured in a strict hierarchical decomposition:

1. **Top Level (High-level Factors):** External quality factors representing management and customer viewpoints (e.g., Maintainability, Reliability).
2. **Second Level (Quality Criteria): Intermediate software attributes representing software engineering viewpoints that directly affect the factors** (e.g., modularity, conciseness, consistency, fault tolerance).
3. **Third Level (Quality Metrics):** Quantitative direct measurements used to calculate the criteria.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 323–324; companion slides 31, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الهرمية في نموذج ماكول: العوامل (Factors) في القمة ← تليها المعايير الهندسية (Criteria) في المستوى الثاني ← تليها المقاييس الكمية المباشرة (Metrics).

Q7.16. Which one is not a level in Boehm software quality model?

(a) Primary uses · (b) Intermediate constructs · (c) Primitive constructs · (d) Final constructs

(d) Final constructs

Barry Boehm's hierarchical quality model (1978) structures quality across three explicit levels:

1. **Primary uses / high-level characteristics:** The figure links General Utility directly to Portability and Maintainability. As-is Utility links to Reliability, Efficiency, and Human Engineering; Maintainability links to Testability, Understandability, and Modifiability.
2. **Intermediate constructs:** Seven constructs, including Portability, Reliability, Efficiency, Human Engineering, Testability, Understandability, and Modifiability.
3. **Primitive constructs:** The model lists 12 lower-level characteristics.

"Final constructs" is not a level in Boehm's taxonomy.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 324–326; companion slides 40, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مستويات نموذج بويم (Boehm) الثلاثة هي: Primary uses ← تليها Intermediate constructs ← ثم Primitive constructs. لا توجد طبقة تسمى «Final constructs».

Q7.17. Which one is not a software product quality model?

(a) McCall model · (b) Boehm model · (c) ISO 9000 · (d) ISO 9126

(c) ISO 9000

McCall's model, Boehm's model, and ISO 9126 are product quality models that evaluate the internal attributes and external runtime characteristics of the software artifact itself.

In contrast, ISO 9000 (and ISO 9001) is a Quality Management System (QMS) standard focused strictly on organizational development processes, management commitment, and procedural compliance, rather than evaluating software product attributes directly.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 321, 354–355; companion slides 29, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- فرق جوهري: نماذج جودة المنتج البرمجي تشمل McCall وBoehm وISO 9126. أما ISO 9000 فهو معيار إدارة عمليات المنشأة (Process Standard) وليس مقياساً لجودة المنتج البرمجي ذاته.

Q7.18. Basic execution time model was developed by:

(a) Bev. Littlewood · (b) J.D. Musa · (c) Barry Boehm · (d) Jim McCall

(b) J.D. Musa

The Basic Execution Time Model was formulated and published by John D. Musa at Bell Laboratories in 1979. It represents one of the foundational Non-Homogeneous Poisson Process (NHPP) software reliability growth models based on execution time.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 329; companion slides 47, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- نموذج زمن التنفيذ الأساسي (Basic Execution Time Model) وضعه جون موسى (J.D. Musa) في مختبرات بيل عام 1979.

Q7.19. NHPP stands for:

(a) Non Homogeneous Poisson Process · (b) Non Heterogeneous Poisson Process · (c) Non Homogeneous Poisson Product · (d) Non Heterogeneous Poisson Product

(a) Non Homogeneous Poisson Process

NHPP stands for Non-Homogeneous Poisson Process. In software reliability growth modeling, an NHPP is a generalized Poisson stochastic process whose failure rate (intensity function $\lambda(t)$) varies dynamically over execution time rather than remaining constant.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 329; companion slides 47, 112.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مصطلح NHPP يعني عملية بواسون غير المتجانسة (Non-Homogeneous Poisson Process)، وتتميز بأن معدل حدوث الأخطاء فيها يتغير مع تقدم زمن الاختبار.

Q7.20. In the Basic execution time model, failure intensity is given by:

(a) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0^2}\right)$ · (b) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$ · (c) $\lambda(\mu) = \lambda_0 \left(1 - \frac{v_0}{\mu}\right)$ · (d) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu^2}{v_0}\right)$

(b) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$

In Musa's Basic Execution Time Model, the failure intensity $\lambda(\mu)$ decreases linearly as a function of the cumulative failures experienced (μ):

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

Where:

- λ_0**: Initial failure intensity at the beginning of execution ($\mu = 0$).**
- v_0**: Total number of failures that would be experienced in infinite execution time.**
- μ**: Cumulative number of failures experienced up to that point.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 330; companion slides 48, 112.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- دالة شدة الفشل في النموذج الأساسي دالة خطية تنازلية: $\lambda(\mu) = \lambda_0(1 - \mu/v_0)$. تنخفض الشدة خطياً مع ازدياد الأعطال المكتشفة والمصلحة.

Q7.21. In the Basic execution time model, the additional number of failures required to achieve a failure intensity objective is expressed as:

(a) $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_F - \lambda_P)$ · (b) $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$ · (c) $\Delta\mu = \frac{\lambda_0}{v_0}(\lambda_P - \lambda_F)$ · (d) $\Delta\mu = \frac{\lambda_0}{v_0}(\lambda_F - \lambda_P)$

(b) $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$

Rearranging the linear failure intensity relationship for the present intensity (λ_P) and objective intensity (λ_F):

$$\lambda_P = \lambda_0\left(1 - \frac{\mu_P}{v_0}\right), \quad \lambda_F = \lambda_0\left(1 - \frac{\mu_F}{v_0}\right)$$

Subtracting the two equations yields:

$$\lambda_P - \lambda_F = \frac{\lambda_0}{v_0}(\mu_F - \mu_P) = \frac{\lambda_0}{v_0}\Delta\mu$$

Solving for $\Delta\mu$ gives:

$$\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 331; companion slides 51, 112.

📝 **Academic Note**

ملاحظة الطالب والفخ الامتحاني:

- عدد الأعطال الإضافية اللازمة للوصول إلى هدف الاعتمادية هو: $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$. انتبه لطرح الهدف المنخفض من الشدة الحالية $(\lambda_P - \lambda_F)$.

Q7.22. In the Basic execution time model, the additional execution time required to achieve a failure intensity objective is given as:

(a) $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_F}{\lambda_P}\right)$ · (b) $\Delta\tau = \frac{\lambda_0}{v_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$ · (c) $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$ · (d) $\Delta\tau = \frac{\lambda_0}{v_0} \ln\left(\frac{\lambda_F}{\lambda_P}\right)$

(c) $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$

In the Basic execution time model, failure intensity decays exponentially with execution time τ:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

Evaluating between current intensity λ_P and target intensity λ_F:

$$\frac{\lambda_P}{\lambda_F} = \exp\left(\frac{\lambda_0}{v_0}\Delta\tau\right)$$

Taking the natural logarithm of both sides and solving for $\Delta\tau$:

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 331; companion slides 52, 113.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- زمن التنفيذ الإضافي المطلوب للوصول لهدف الاختبار: $\Delta\tau = \frac{v_0}{\lambda_0} \ln(\lambda_P/\lambda_F)$. النسبة داخل اللوغاريتم هي القيمة الأكبر على الأصغر (λ_P/λ_F) لضمان ناتج موجب.

Q7.23. The failure intensity function of the Logarithmic Poisson execution time model is given as:

(a) $\lambda(\mu) = \lambda_0 \ln(-\theta\mu)$ · (b) $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$ · (c) $\lambda(\mu) = \lambda_0 \exp(\theta\mu)$ · (d) $\lambda(\mu) = \lambda_0 \log(\theta\mu)$

(b) $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$

In the Logarithmic Poisson execution time model (Musa–Okumoto, 1984), failure intensity decreases exponentially with respect to cumulative failures experienced (μ):

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

Where θ is the failure intensity decay parameter. This exponential decay captures the realistic observation that early faults contribute much more heavily to failure intensity than subtle faults discovered late in testing.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 335; companion slides 59, 113.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- دالة شدة الفشل في نموذج بواسون اللوغاريتمي تنخفض أسياً مع عدد الأخطاء: $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$.

Q7.24. In the Logarithmic Poisson execution time model, "θ" is known as:

(a) Failure intensity function parameter · (b) Failure intensity decay parameter · (c) Failure intensity measurement · (d) Failure intensity increment parameter

(b) Failure intensity decay parameter

In the Logarithmic Poisson formulation, θ is explicitly termed the failure intensity decay parameter (measured in failure^{-1}). It quantifies the relative reduction in failure intensity per failure experienced:

$$\theta = -\frac{d\lambda/d\mu}{\lambda}$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 335; companion slides 59, 113.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الرمز θ في نموذج بواسون اللوغاريتمي يمثل معامل تلاشي شدة الفشل (Failure intensity decay parameter).

Q7.25. In the Jelinski–Moranda model, failure intensity is defined as:

(a) $\lambda(t_i) = \phi[N - (i-1)]$ · (b) $\lambda(t_i) = \phi[N + (i-1)]$ · (c) $\lambda(t_i) = \phi[N - i]$ · (d) $\lambda(t_i) = \phi[N/i]$

(a) $\lambda(t_i) = \phi[N - (i-1)]$

The Jelinski–Moranda model (1972) assumes that software contains an initial total of N faults, each contributing an equal hazard rate ϕ. When the i-th failure occurs, $(i-1)$ faults have already been completely and perfectly removed without introducing new faults. Thus, during the interval between the $(i-1)$-th and i-th failure, the failure intensity is:

$$\lambda(t_i) = \phi[N - (i-1)]$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 345; companion slides 90, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- معادلة جيلينسكي-موراندا: شدة الفشل عند العطل رقم i تتناسب مع عدد الأخطاء المتبقية: $\phi[N - (i-1)]$.

Q7.26. CMM Level 1 has:

(a) 6 KPAs · (b) 2 KPAs · (c) 0 KPAs · (d) None of the above

(c) 0 KPAs

In the Capability Maturity Model (CMM), Level 1 (Initial) has exactly zero Key Process Areas (KPAs). At Level 1, processes are ad-hoc, chaotic, and undefined; project success depends solely on individual heroism and crisis management rather than repeatable institutional practices. KPAs appear beginning at Level 2 (which has 6 KPAs).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 348; companion slides 95, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- المستوى الأول في نموذج CMM (المستوى الابتدائي Initial) يحتوي على **صفر من مناطق العمليات الرئيسية (0 KPAs)** لأن العمل فيه فوضوي وغير خاضع لمعايير مؤسسية.

Q7.27. MTBF stands for:

(a) Mean time between failure · (b) Maximum time between failures · (c) Minimum time between failures · (d) Many time between failures

(a) Mean time between failure

MTBF stands for Mean Time Between Failures. It represents the average elapsed operational time separating successive system failures in repairable systems, calculated as the sum of Mean Time To Failure (MTTF) and Mean Time To Repair (MTTR):

$$MTBF = MTTF + MTTR$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 312; companion slides 13, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مصطلح MTBF يعني متوسط الزمن بين الانهيارات (Mean Time Between Failures) في الأنظمة القابلة للإصلاح.

Q7.28. CMM model is a technique to:

(a) Improve the software process · (b) Automatically develop the software · (c) Test the software · (d) All of the above

(a) Improve the software process

The Capability Maturity Model (SEI CMM) is an organizational framework designed specifically to assess the maturity of software processes and guide organizations through evolutionary stages of software process improvement. It does not automatically generate source code or function as an automated testing engine.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- هدف CMM الأساسي هو تحسين نضج العملية البرمجية للمؤسسة (Improve the software process)، وليس توليد كود آلياً.

Q7.29. Total number of maturity levels in CMM are:

(a) 1 · (b) 3 · (c) 5 · (d) 7

(c) 5

The SEI Capability Maturity Model defines exactly five evolutionary maturity levels:

1. **Level 1 — Initial:** Chaotic, ad-hoc performance.
2. **Level 2 — Repeatable:** Basic project management controls.
3. **Level 3 — Defined:** Standardized, institutionalized engineering processes.
4. **Level 4 — Managed:** Quantitative process measurement and product quality metrics.
5. **Level 5 — Optimizing:** Continuous process improvement through feedback and technological innovation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 348; companion slides 95, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مستويات CMM الخمسة بالترتيب: 1- Initial ← 2- Repeatable ← 3- Defined ← 4- Managed ← 5- Optimizing.

Q7.30. Reliability of a software is dependent on number of errors:

(a) Removed · (b) Remaining · (c) Both (a) & (b) · (d) None of the above

(b) Remaining

Software reliability is determined strictly by the latent faults currently remaining in the code that can be encountered during execution, combined with the frequency of their activation under the operational profile. The historical count of faults removed during past debugging sessions does not dictate current system stability; a system from which 1,000 faults were removed may still fail immediately if critical residual faults remain in frequently executed pathways.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 309, 313; companion slides 9, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- فخ امتحاني شهير: اعتمادية النظام تعتمد حصراً على عدد الأخطاء المتبقية (Remaining) في الكود أثناء التشغيل، ولا تعتمد على عدد الأخطاء التي تم إصلاحها في الماضي.

Q7.31. Reliability of software is usually estimated at:

(a) Analysis phase · (b) Design phase · (c) Coding phase · (d) Testing phase

(d) Testing phase

Dynamic software reliability models (such as Musa's models and Jelinski–Moranda) require executing the software under an operational profile and collecting dynamic failure timestamps and counts. Consequently, statistical estimation and measurement of software reliability take place primarily during system and integration **testing**, once an executable system exists.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 312–313; companion slides 10, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تقدير الاعتمادية إحصائياً يحتاج إلى تشغيل الكود وتسجيل أزمنة الفشل، ولذلك يتم قياسها في مرحلة الاختبار (Testing phase).

Q7.32. CMM stands for:

(a) Capacity maturity model · (b) Capability maturity model · (c) Cost management model · (d) Comprehensive maintenance model

(b) Capability maturity model

CMM stands for Capability Maturity Model, developed by the Software Engineering Institute (SEI) at Carnegie Mellon University.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- CMM اختصار لـ Capability Maturity Model.

Q7.33. Which level of CMM is for basic project management?

(a) Initial · (b) Repeatable · (c) Defined · (d) Managed

(b) Repeatable

Level 2 (Repeatable) of the Capability Maturity Model establishes fundamental project management controls, focusing on project planning, requirements management, project tracking, subcontract management, quality assurance, and configuration management to ensure realistic commitments based on past project successes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 348; companion slides 95, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- المستوى الثاني (Repeatable) هو المستوى المسؤول عن إدارة المشاريع الأساسية (Basic project management).

Q7.34. Which level of CMM is for process control?

(a) Initial · (b) Repeatable · (c) Defined · (d) Managed

(d) Managed

Level 4 (Managed) uses quantitative measures to control process and product quality.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 361; companion slide 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- ضبط والتحكم في العمليات (Process control) كمياً هو ميزة المستوى الرابع (Managed).
- الشريحة 115 تكرر صياغة سؤال 7.35؛ صيغة هذا السؤال عن Process control.

Q7.35. Which level of CMM is for process management?

(a) Initial · (b) Defined · (c) Managed · (d) Optimizing

(b) Defined

At Level 3 (Defined), standard software engineering and management processes are documented, standardized, and integrated into an organization-wide standard software process (institutional process management). Projects tailor these standard organizational processes to develop approved, managed project processes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 348–349, 361; companion slide 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- توثيق وإدارة العمليات المؤسسية القياسية (Process management) هو عنوان المستوى الثالث (Defined).

Q7.36. CMM was developed at:

(a) Harvard University · (b) Cambridge University · (c) Carnegie Mellon University · (d) Maryland University

(c) Carnegie Mellon University

The Capability Maturity Model (CMM) was developed by the Software Engineering Institute (SEI), a federally funded research and development center established at Carnegie Mellon University (CMU) in Pittsburgh, Pennsylvania.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- طور معهد SEI نموذج CMM في جامعة كارنيغي ميلون (Carnegie Mellon University).

Q7.37. McCall has developed a:

(a) Quality model · (b) Process model · (c) Reliability model · (d) None of the above

(a) Quality model

Jim McCall and his associates developed a landmark Software Quality Model in 1977 for the US Air Force, pioneering the decomposition of software quality into factors, criteria, and metrics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 30, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- وضع ماكول (McCall) نموذجاً لجودة البرمجيات (Quality model).

Q7.38. The model to measure software process improvement is called:

(a) ISO 9000 · (b) ISO 9126 · (c) CMM · (d) Spiral model

(c) CMM

The Capability Maturity Model (CMM) provides a structured metric and staging framework specifically designed to evaluate organizational process capability and track continuous process improvement. ISO 9126 measures product quality, ISO 9000 is a baseline certification standard, and the Spiral model is a life-cycle methodology.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الموديل المخصص لقياس تحسين نضج العملية البرمجية وتدرجها هو CMM.

Q7.39. The number of clauses used in ISO 9001 are:

(a) 15 · (b) 25 · (c) 20 · (d) 10

(c) 20

ISO 9001:1994 organized its quality-system requirements into 20 clauses, numbered 4.1 through 4.20.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 355; companion slides 103, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- اعتمدت مواصفة ISO 9001 (إصدار 1994 المنهجي في الكتاب) على **20 بنداً أو فقرة متطلبات (clauses 20)** تم ربطها مع متطلبات CMM.

Q7.40. ISO 9126 contains definitions of:

(a) Quality characteristics · (b) Quality factors · (c) Quality attributes · (d) All of the above

(a) Quality characteristics

The ISO/IEC 9126 standard formally structures software product quality into six top-level quality characteristics: Functionality, Reliability, Usability, Efficiency, Maintainability, and Portability. Each characteristic is further refined into subcharacteristics and measurable metrics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 326–327; companion slides 41, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- في معيار ISO 9126، يسمى المستوى الأعلى بخصائص الجودة (Quality characteristics)، وعددها 6 خصائص رئيسية.

Q7.41. In ISO 9126, each characteristic is related to:

(a) One attribute · (b) Two attributes · (c) Three attributes · (d) Four attributes

No single option is correct for all six characteristics. The listed ISO 9126 table gives different numbers of subcharacteristics: Reliability has three, Efficiency has two, and Functionality has four.

The MCQ is therefore inconsistent as worded.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 327; companion slides 41, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- يرتبط كل بعد رئيسي في سلايدات أغروال بصفات فرعية محددة (Subcharacteristics) تمثل السمات المشتقة.
- الشريحة 117 تضع الخيار (c)، لكن جدول الكتاب يبيّن أن العدد يختلف حسب الخاصية؛ لذلك لا يوجد خيار واحد يصح للجميع.

Q7.42. In McCall quality model, product revision quality factor consists of:

(a) Maintainability, Flexibility, Testability · (b) Portability, Reusability, Interoperability · (c) Correctness, Reliability, Efficiency · (d) Usability, Integrity, Efficiency

(a) Maintainability, Flexibility, Testability

Under McCall's framework, Product Revision focuses on how easily the software can be inspected, repaired, and enhanced. It consists of exactly three factors:

1. **Maintainability:** Effort required to locate and fix defects.
2. **Flexibility:** Effort required to modify the software for new operational needs.
3. **Testability:** Effort required to validate that the software performs its intended functions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 33, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- عوامل مراجعة المنتج (Product Revision) في نموذج ماكول هي ثلاثية الصيانة: سهولة الصيانة (Maintainability) + المرونة (Flexibility) + القابلية للاختبار (Testability).

Q7.43. Which is not a software reliability model?

(a) The Jelinski–Moranda Model · (b) Basic execution time model · (c) Spiral model · (d) None of the above

(c) Spiral model

The Spiral model is an iterative, risk-driven Software Development Life Cycle (SDLC) process model formulated by Barry Boehm. It is not a statistical software reliability growth model. Jelinski–Moranda and Musa's Basic execution time model are established reliability models.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 329, 345; companion slides 47, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- النموذج الحلزوني (Spiral model) هو نموذج دورة حياة لتطوير البرمجيات (SDLC) يعتمد على إدارة المخاطر، وليس نموذجاً رياضياً لقياس الاعتمادية.

Q7.44. Each maturity level in CMM has:

(a) One KPA · (b) Equal KPAs · (c) Several KPAs · (d) No KPA

(c) Several KPAs

With the exception of Level 1 (which has 0 KPAs), each mature level in the CMM comprises multiple Key Process Areas (KPAs): Level 2 has 6 KPAs, Level 3 has 7 KPAs, Level 4 has 2 KPAs, and Level 5 has 3 KPAs.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 350–352; companion slides 97–101, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- يضم كل مستوى من مستويات CMM الناضجة عدة مناطق عمليات رئيسية (Several KPAs)، تتراوح بين 2 و7 مجالات.

Q7.45. KPA in CMM stands for:

(a) Key Process Area · (b) Key Product Area · (c) Key Principal Area · (d) Key Performance Area

(a) Key Process Area

KPA stands for Key Process Area. In CMM, a Key Process Area identifies a cluster of related activities that, when performed collectively, achieve a set of goals considered critical for establishing process capability at that maturity level.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 350; companion slides 97, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مصطلح KPA يعني منطقة العملية الرئيسية (Key Process Area).

Q7.46. In reliability models, our emphasis is on:

(a) Errors · (b) Faults · (c) Failures · (d) Bugs

(c) Failures

Dynamic software reliability models model the observable behavior of the software from the user's perspective; hence, the primary modeling emphasis is on failures (dynamic departure from required operation). Users experience failures, not passive latent faults residing silently in unexecuted source code.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 310, 329; companion slides 8, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تركز نماذج الاعتمادية على حالات الفشل (Failures) لأنها الحدث الديناميكي الحقيقي الذي يواجهه المستخدم ويؤثر على الخدمة في الواقع.

Q7.47. Software does not break or wear out like hardware. What is your opinion?

(a) True · (b) False · (c) Partially True · (d) Cannot say

(a) True

True in the physical sense: software does not wear out through physical aging. It may become obsolete or fail when its environment or requirements change, but those changes are different from hardware wear-out.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 308; companion slides 4, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- العبارة صحيحة تماماً (True): البرمجيات لا تهترئ ولا تبلى مادياً (Does not wear out) لأنها كيان منطقي غير خاضع لقوانين الاحتكاك والفيزياء.

Q7.48. Software reliability is defined with respect to:

(a) Time · (b) Speed · (c) Quality · (d) None of the above

(a) Time

Software reliability is mathematically defined as a time-dependent probability distribution $R(t)$ measuring the likelihood of failure-free operation over a specified operational time interval. Without specifying a temporal baseline (execution time, clock time, or calendar time), reliability has no mathematical meaning.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 309; companion slides 6, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- دالة الاعتمادية تُعرّف رياضياً بالنسبة إلى الزمن (Time).

Q7.49. MTTF stands for:

(a) Mean time to failure · (b) Maximum time to failure · (c) Minimum time to failure · (d) None of the above

(a) Mean time to failure

MTTF stands for Mean Time To Failure. It is the expected statistical value of the operational execution time of a software system before its next failure event occurs, defined as the integral of the reliability function:

$$MTTF = \int_0^\infty R(t)\,dt$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 312; companion slides 13, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- اختصار MTTF يعني متوسط زمن حدوث الفشل (Mean Time To Failure).

Q7.50. ISO 9000 is a series of standards for quality management systems and has:

(a) 2 related standards · (b) 5 related standards · (c) 10 related standards · (d) 25 related standards

(b) 5 related standards

The original ISO 9000 series comprises five related standards:

1. **ISO 9000:** Quality management and quality assurance standards — Guidelines for selection and use.
2. **ISO 9001:** Model for quality assurance in design, development, production, installation, and servicing.
3. **ISO 9002:** Model for quality assurance in production, installation, and servicing.
4. **ISO 9003:** Model for quality assurance in final inspection and test.
5. **ISO 9004:** Quality management and quality system elements — Guidelines.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 354; companion slides 103, 119.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تتكون سلسلة معايير ISO 9000 الأصلية من **5 مواصفات مرتبطة** شهيرة (ISO 9000, 9001, 9002, 9003, 9004).

Section 2: Syllabus Gap-Closing Synthesis MCQs (10 Items)

Q7.51. In Harlan Mills' Bug Seeding Model, if S artificial faults are seeded into a program, and testing subsequently uncovers s seeded faults and n original unseeded faults, the estimated number of latent unseeded faults $\hat{N}$ is:

(a) $\hat{N} = \frac{S \cdot n}{s}$ · (b) $\hat{N} = \frac{s \cdot n}{S}$ · (c) $\hat{N} = \frac{S \cdot s}{n}$ · (d) $\hat{N} = S \cdot n \cdot s$

(a) $\hat{N} = \frac{S \cdot n}{s}$

The Bug Seeding (or Tag-Recapture) model developed by Harlan Mills assumes that seeded artificial faults and intrinsic unseeded faults have equal probability of discovery. Therefore, the ratio of caught seeded faults to total seeded faults equals the ratio of caught unseeded faults to total unseeded faults:

$$\frac{s}{S} = \frac{n}{\hat{N}} \implies \hat{N} = \frac{S \cdot n}{s}$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 346–347; companion slide 93.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- معادلة بذر الأخطاء (Bug Seeding) لميلز تعتمد على التناسب المتساوي لصيد الأخطاء: $\hat{N} = (S \times n)/s$.

Q7.52. In software reliability engineering, what is an "Operational Profile"?

(a) The compiler optimization configuration · (b) A quantitative set of operational run types and their associated probabilities of occurrence · (c) The hardware specification of the deployment server · (d) The developer's daily work schedule

(b) A quantitative set of operational run types and their associated probabilities of occurrence

An Operational Profile is a complete set of operations that a software system performs, together with their individual probabilities of occurrence during actual field usage. Testing according to the operational profile gives more testing effort to frequently executed operations.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 318–320; companion slides 16–20.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- ملف التشغيل (Operational Profile) يحدد أنواع العمليات في النظام مع احتمالية تشغيل كل منها في الواقع، لتوجيه الاختبار نحو المسارات الأكثر استخداماً.

Q7.53. In Musa's Calendar Time Component modeling, during which operational phase does testing progress slow down primarily due to personnel limitations rather than compute constraints?

(a) Test-limiting phase · (b) Failure-identification personnel limited phase · (c) Data-entry limiting phase · (d) Overclocked phase

(b) Failure-identification personnel limited phase

Musa's Calendar Time Component models testing progress under three resource constraints:

1. **Failure identification personnel:** Bounded by available system test engineers.
2. **Failure resolution personnel:** Bounded by development debugging staff.
3. **Computer execution resources:** Bounded by available CPU test-bench time.

Early in testing when failure intensity is very high, progress is constrained strictly by the finite hours that test and debugging personnel can devote to analyzing crashes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 339–344; companion slides 72–77.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مكون الزمن التقويمي (Calendar time) يقسم الموارد إلى: موظفي تشخيص الفشل، وموظفي الإصلاح، وزمن المعالج. في بداية الاختبار عندما تتساقط الأخطاء بكثرة، يكون المورد الحاكم المقيد هو كادر التحقيق والإصلاح.

Q7.54. In the Capability Maturity Model (CMM), what are the "Common Features" that characterize every Key Process Area?

(a) Input, Process, Output, Storage · (b) Commitment to Perform, Ability to Perform, Activities Performed, Measurement & Analysis, Verifying Implementation · (c) Goals, Metrics, Actions, Feedback · (d) Planning, Execution, Review, Retirement

(b) Commitment to Perform, Ability to Perform, Activities Performed, Measurement & Analysis, Verifying Implementation

In SEI CMM, each KPA is organized across five structural Common Features:

1. **Commitment to Perform:** Organizational policies and management leadership.
2. **Ability to Perform:** Resources, funding, tools, and training preconditions.
3. **Activities Performed:** Specific engineering and management tasks.
4. **Measurement and Analysis:** Basic metrics used to track process performance.
5. **Verifying Implementation:** Objective reviews and audits by SQA and executive management.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 353–354; companion slide 102.

📝 **Academic Note**

ملاحظة الطالب والفخ الامتحاني:

- السمات المشتركة (Common Features) الخمس التي تبني أي KPA في CMM هي: الالتزام بالأداء ← القدرة على الأداء ← الأنشطة المنفذة ← القياس والتحليل ← التحقق من التنفيذ.

Q7.55. What is the main difference in emphasis between ISO 9001 and CMM?

(a) ISO 9001 emphasizes continuous software-process improvement; CMM defines only minimum quality-system criteria · (b) CMM emphasizes continuous process improvement, while ISO 9001 defines minimum criteria for an adequate quality-management system · (c) ISO 9001 applies only to software; CMM applies to all industries · (d) CMM concerns documentation only, while ISO 9001 provides a staged software-improvement path

(b) CMM emphasizes continuous process improvement, while ISO 9001 defines minimum criteria for an adequate quality-management system

CMM places greater emphasis on continuous process improvement. ISO 9001 defines minimum criteria for an adequate quality-management system.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.5.2–§7.5.3, pp. 356–357; companion slide 107.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الفرق المحوري هو أن CMM يركز أكثر على التحسين المستمر للعمليات، بينما يحدد ISO 9001 الحد الأدنى لنظام إدارة جودة مقبول.

Q7.56. In McCall's Software Quality Model, which factor belongs to the "Product Transition" dimension?

(a) Correctness · (b) Flexibility · (c) Portability · (d) Usability

(c) Portability

Product Transition encompasses three factors concerned with moving the software to new operating environments: Portability, Reusability, and Interoperability. Correctness and Usability belong to Product Operation; Flexibility belongs to Product Revision.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 30–35.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- عوامل الانتقال (Product Transition) الثلاثة في ماكول هي: القابلية للنقل (Portability) وإعادة الاستخدام (Reusability) والتشغيل البيني (Interoperability).

Q7.57. In the Logarithmic Poisson execution time model, how does total failures experienced behave as execution time $\tau \to \infty$?

(a) It reaches a finite upper limit v_0 · (b) It grows logarithmically without an infinite upper bound ($\mu \to \infty$) · (c) It drops immediately to zero · (d) It becomes negative

(b) It grows logarithmically without an infinite upper bound ($\mu \to \infty$)

A distinguishing mathematical attribute of the Musa–Okumoto Logarithmic Poisson model is that it models an infinite-failure category:

$$\mu(\tau) = \frac{1}{\theta} \ln(\lambda_0 \theta \tau + 1)$$

As $\tau \to \infty$, $\mu(\tau) \to \infty$ logarithmically. Unlike the Basic model (which posits a finite total fault count v_0), the Logarithmic Poisson model assumes that software will continue to generate failures indefinitely at increasingly tiny failure intensities as software complexity and operational duration grow unbounded.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 335–336; companion slides 59–60.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- فرق جوهري بين النموذجين: النموذج الأساسي يفترض سقفاً نهائياً لعدد الأعطال (v_0)، بينما نموذج بواسون اللوغاريتمي يفترض أن الأعطال المتوقعة تتزايد لوغاريتمياً دون سقف أعلى محدود مع الزمن اللانهائي.

Q7.58. Which Key Process Area (KPA) belongs to CMM Level 2 (Repeatable)?

(a) Organization Process Definition · (b) Software Configuration Management (SCM) · (c) Quantitative Process Management · (d) Defect Prevention

(b) Software Configuration Management (SCM)

Level 2 (Repeatable) comprises exactly six KPAs focused on project-level tracking and discipline:

1. Requirements Management
2. Software Project Planning
3. Software Project Tracking and Oversight
4. Software Subcontract Management
5. Software Quality Assurance
6. **Software Configuration Management**

Organization Process Definition is Level 3; Quantitative Process Management is Level 4; Defect Prevention is Level 5.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 350; companion slides 97, 101.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مناطق العمليات الست للمستوى الثاني (Level 2): إدارة المتطلبات، تخطيط المشاريع، التتبع والرقابة، إدارة التعاقدات الفرعية، ضمان جودة البرمجيات (SQA)، وإدارة التكوين (SCM).

Q7.59. The relationship between execution time τ and calendar time t in software reliability is mediated by:

(a) The compiler execution clock · (b) Resource allocation parameters representing available personnel, compute time, and their utilization ratios · (c) The lines of source code · (d) Function point complexity adjustment

(b) Resource allocation parameters representing available personnel, compute time, and their utilization ratios

Execution time measures CPU usage; calendar time also depends on the resources available and how fully each resource can be utilized. The limiting resource determines the calendar-time to execution-time ratio:

$$\frac{dt}{d\tau} = \max_{r \in \{I,F,C\}} \left(\frac{1}{P_r p_r} \frac{d\chi_r}{d\tau} \right) = \max_{r \in \{I,F,C\}} \left(\frac{\theta_r + \mu_r \lambda}{P_r p_r} \right)$$

Here, P_r is the available quantity of resource r, p_r is its utilization fraction, and χ_r is its cumulative usage. The terms θ_r and μ_r are resource usage per CPU hour and per failure, respectively; λ is failure intensity.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 339–344; companion slides 72–77.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- التحويل من زمن التنفيذ (τ) إلى الزمن التقويمي (t) يتحكم به توفر الموارد البشرية (كادر الفحص وكادر الإصلاح) وقدرات أجهزة المعالجة.

Q7.60. In CMM process maturity staging, which level introduces statistical process control (SPC) and quantitative process capability baselines?

(a) Level 2 · (b) Level 3 · (c) Level 4 · (d) Level 5

(c) Level 4

Level 4 (Managed) introduces quantitative management and statistical process control (SPC). Productivity and quality metrics are measured across all process activities, establishing statistical process capability baselines that allow management to distinguish common cause variation from special cause variation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 348–349; companion slides 95, 100.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الضبط الإحصائي للعمليات (Statistical Process Control) والقياس الكمي الدقيق يبدأ حصراً في المستوى الرابع (Level 4: Managed).

ASE CHAPTER 8

T1 Multiple-Choice Questions Pack

Q8.1. Software testing is:

(a) the process of demonstrating that errors are not present · (b) the process of establishing confidence that a program does what it is supposed to do · (c) the process of executing a program to show it is working as per specifications · (d) the process of executing a program with the intent of finding errors

(d) the process of executing a program with the intent of finding errors

Software testing is defined classically by Myers (1979) as the process of executing a program with the intent of finding errors. Testing demonstrates the presence of defects rather than their absolute absence.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.1.1 (p. 365); slide 3; question on slide 189.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

تعريف مايرز الكلاسيكي (Glenford Myers, 1979): الاختبار هو عملية تنفيذ البرنامج بقصد اكتشاف الأخطاء والعيوب الكامنة، وليس إثبات خلوه التام من الأخطاء لأن الاختبار الشامل مستحيل حسابياً.

Q8.2. Software mistakes during coding are known as:

(a) failures · (b) defects · (c) bugs · (d) errors

(c) bugs

When developers make mistakes while coding, these mistakes are called bugs. People make errors (a good synonym is mistake); a failure occurs only when a fault executes.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.1 (p. 369); slide 9; question on slide 189.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

الخطأ البشري أثناء كتابة الكود يسمى «bug». لا تخلط بين الخيارين: errors هو الفعل البشري العام، أما bugs فهو الاسم الذي يطلقه المصدر على أخطاء الترميز.

Q8.3. Functional testing is known as:

(a) Structural testing · (b) Behavior testing · (c) Regression testing · (d) None of the above

(b) Behavior testing

Functional testing is known as behavioral testing or black-box testing. It treats the software component as a black box, deriving test inputs purely from requirements specifications without examining internal structural control flow.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3 (p. 371); slide 13; question on slide 189.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

الفحص الوظيفي (Functional Testing) يُعرف أيضاً بالفحص السلوكي (Behavioral Testing) أو فحص الصندوق الأسود (Black-box)؛ حيث تُبنى حالات الاختبار استناداً إلى المدخلات والمخرجات المطلوبة بالمواصفات دون النظر للكود الداخلي.

Q8.4. For a function of n variables, boundary value analysis yields:

(a) 4n+3 test cases · (b) 4n+1 test cases · (c) n+4 test cases · (d) None of the above

(b) 4n+1 test cases

For a function of n variables, boundary value analysis (BVA) yields $4n + 1$ test cases. Under the single-fault assumption, each variable is tested at four boundary extremes ($min, min+, max-, max$) while holding other variables at nominal, plus one common baseline test case with all variables at nominal values: $4(n) + 1 = 4n + 1$.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.1 (p. 372); slides 15, 20; question on slide 189.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

تحليل القيم الحدية القياسي (BVA) يولد $4n + 1$ حالة اختبار لعدد n من المتغيرات اعتماداً على فرضية العيب المنفرد (Single-fault assumption)، حيث يُختبر كل متغير عند 4 قيم حدية وتبقى بقية المتغيرات عند قيمتها الطبيعية (Nominal)، بالإضافة لحالة اختبار مركزية واحدة.

Q8.5. For a function of two variables, how many cases will be generated by robustness testing?

(a) 9 · (b) 13 · (c) 25 · (d) 42

(b) 13

For a function of two variables ($n = 2$), robustness testing generates $6n + 1 = 6(2) + 1 = 13$ test cases. Robustness testing adds values just outside the valid boundaries ($min-$ and $max+$), testing 6 extreme points per variable plus the nominal center.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.1 (p. 372); slides 24–25; question on slide 190.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

فحص المتانة (Robustness Testing) يختبر القيم الواقعة خارج الحدود مباشرة ($min-$ و $max+$) بالإضافة للحدود الأربعة، فيكون لكل متغير 6 قيم حدية بالإضافة للحالة الطبيعية ($6n + 1$). لمتغيرين اثنين: $6(2) + 1 = 13$ حالة اختبار.

Q8.6. For a function of n variables robustness testing of boundary value analysis yields:

(a) 4n+1 · (b) 4n+3 · (c) 6n+1 · (d) None of the above

(c) 6n+1

For a function of n variables, robustness testing of boundary value analysis yields $6n + 1$ test cases. Each variable takes values $\{min-, min, min+, nom, max-, max, max+\}$, producing 6 tests per variable held against nominal values of all other inputs, plus 1 all-nominal test case.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.1 (p. 372); slide 24; question on slide 190.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

قاعدة عدد حالات اختبار المتانة العامة لـ n متغير هي $6n + 1$، وهي امتداد لقاعدة BVA القياسية ($4n + 1$) بإضافة نقطتين غير صالحتين لكل متغير لاختبار مناعة النظام ضد المدخلات غير المقبولة.

Q8.7. Regression testing is primarily related to:

(a) Functional testing · (b) Data flow testing · (c) Development testing · (d) Maintenance testing

(d) Maintenance testing

Publisher key: regression testing is primarily related to maintenance testing, meaning re-testing after software has been changed.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; no explanatory slide; publisher key; question on slide 190.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

اختبار الانحدار (Regression) يرتبط بمرحلة الصيانة: إعادة الفحص بعد أي تعديل. السلايدات لا تشرح هذا البند بجملة صريحة، فاحفظه كمفتاح الناشر.

Q8.8. A node with indegree=0 and out degree ≠0 is called

(a) Source node · (b) Destination node · (c) Transfer node · (d) None of the above

(a) Source node

In a control flow graph, a node with indegree $= 0$ and outdegree $\neq 0$ is called a source node. It represents the entry point of the program or module.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4 (p. 406); flow-graph background, slide 105; question on slide 190.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

في مخطط تدفق التحكم (Control Flow Graph)، العقدة التي لا يدخل إليها أي سهم ($indegree = 0$) ويخرج منها مسار واحد أو أكثر ($outdegree \neq 0$) تُسمى عقدة المصدر أو البداية (Source Node).

Q8.9. A node with indegree ≠0 and out degree=0 is called

(a) Source node · (b) Predicate node · (c) Destination node · (d) None of the above

(c) Destination node

In a control flow graph, a node with indegree $\neq 0$ and outdegree $= 0$ is called a destination node (or sink node). It represents the exit point or termination of the control flow.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4 (p. 406); flow-graph background, slide 105; question on slide 190.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

العقدة التي تدخل إليها مسارات ($indegree \neq 0$) ولا يخرج منها أي مسار ($outdegree = 0$) هي عقدة المقصد أو النهاية (Destination / Sink Node).

Q8.10. A decision table has

(a) Four portions · (b) Three portions · (c) Five portions · (d) Two portions

(a) Four portions

A decision table has four distinct portions: (1) Condition Stub in the upper left, (2) Condition Entry in the upper right, (3) Action Stub in the lower left, and (4) Action Entry in the lower right.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.3 (p. 395); slide 77; question on slide 191.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

جدول القرار (Decision Table) يتكون بنيوياً من 4 أرباع محددة: جذع الشروط (أعلى اليسار)، بنود الشروط (أعلى اليمين)، جذع الإجراءات (أسفل اليسار)، وبنود الإجراءات (أسفل اليمين).

Q8.11. Beta testing is carried out by

(a) Users · (b) Developers · (c) Testers · (d) All of the above

(a) Users

Beta testing is carried out by real end users or customers at external customer sites. It represents live operational evaluation in unconstrained, real-world user environments without active intervention from the development team.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slide 12; question on slide 191.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

اختبار بيتا (Beta testing) ينفذه المستخدمون الفعليون (Users/Customers) في مواقعهم وبيئات عملهم الواقعية دون رقابة مباشرة من فريق التطوير.

Q8.12. Equivalence class partitioning is related to

(a) Structural testing · (b) Blackbox testing · (c) Mutation testing · (d) All of the above

(b) Blackbox testing

Equivalence class partitioning is a black-box (functional) technique: the input domain is divided into classes from the specification, and test cases are chosen from each class without looking at the code.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.2 (p. 390); slides 61–62; question on slide 191.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

تجزئة فئات التكافؤ تقنية صندوق أسود (Black-box) لأنها تُشتق من المواصفات فقط. الفخ: الخياران Structural وMutation يخصان عائلات أخرى، وليسا صحيحين.

Q8.13. Cause effect graphing techniques is one form of

(a) Maintenance testing · (b) Structural testing · (c) Function testing · (d) Regression testing

(c) Function testing

Cause-effect graphing is one form of functional (black-box) testing technique. It translates complex natural-language specifications into Boolean logical graphs relating input conditions (causes) to output responses (effects) to systematically generate decision tables.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.4 (p. 402); slide 92; question on slide 191.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

مخطط السبب والنتيجة (Cause-Effect Graphing) هو تقنية متقدمة تابعة لفحوصات الصندوق الأسود الوظيفية (Functional Testing)، تهدف لتحويل العلاقات المنطقية بين المدخلات والمخرجات إلى جداول قرار دقيقة.

Q8.14. During validation

(a) Process is checked · (b) Product is checked · (c) Developer's performance is evaluated · (d) The customer checks the product

(d) The customer checks the product

During validation the customer checks the product. Alpha, beta and acceptance testing are the ways of involving the customer during testing, and the product is judged against the customer's expectations.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.3 (p. 370); slides 11, 178; question on slide 191.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

في Validation الزبون هو من يفحص المنتج (Alpha وBeta وAcceptance كلها طرق لإشراك الزبون). الفخ: الخيار (b) Product is checked صحيح لغوياً لكنه لا يميّز Validation، ومفتاح الناشر هو (d).

Q8.15. Verification is

(a) Checking the product with respect to customer's expectation · (b) Checking the product with respect to specifications · (c) Checking the product with respect to the constraints of the project · (d) All of the above

(b) Checking the product with respect to specifications

Verification is checking the product with respect to its specifications: evaluating whether the products of a development phase satisfy the conditions imposed at the start of that phase.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.3 (p. 370); slide 11; question on slide 192.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

Verification: مطابقة ناتج كل مرحلة مع المواصفات والشروط التي وُضعت في بدايتها. أما مطابقة توقعات الزبون فهي Validation.

Q8.16. Validation is

(a) Checking the product with respect to customer's expectation · (b) Checking the product with respect to specifications · (c) Checking the product with respect to the constraints of the project · (d) All of the above

(a) Checking the product with respect to customer's expectation

Validation is checking the product with respect to the customer's expectation. The deck words it as evaluating the system during or at the end of development against the specified requirements; the key chooses the customer-expectation option.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.3 (p. 370); slides 11, 176; question on slide 192.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

Validation: مطابقة المنتج لتوقعات الزبون. تنبيه: صياغة السلايد 11 هي «specified requirements»، لكن مفتاح الناشر لهذا السؤال هو (a) فاعتمده.

Q8.17. Alpha testing is done by

(a) Customer · (b) Tester · (c) Developer · (d) All of the above

(a) Customer

Alpha tests are conducted at the developer's site by some potential customers, in a controlled environment. The customer is therefore the keyed answer.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slide 12; question on slide 192.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

اختبار ألفا ينفذه زبائن محتملون في موقع المطور وبيئة متحكَّم بها. الفخ: لا تختر Developer لأن المفتاح (a) Customer.

Q8.18. Site for Alpha testing is

(a) Software company · (b) Installation place · (c) Any where · (d) None of the above

(a) Software company

The site for Alpha testing is the developer's site. It takes place in a controlled development setting where developers can directly monitor runtime behavior and immediately record diagnostic anomaly traces.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slide 12; question on slide 192.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

الموقع الحصري لاختبار ألفا هو موقع المطور (Developer's site) وبيئته الداخلية المتحكم بها.

Q8.19. Site for Beta testing is

(a) Software company · (b) User's site · (c) Any where · (d) All of the above

(b) User's site

The site for Beta testing is the user's site. Software is distributed to real end users who execute it within their diverse, unmonitored production hardware and operating configurations.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slide 12; question on slide 193.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

موقع اختبار بيتا هو موقع المستخدم الفعلي (User's site) لضمان العمل على البيئات والأجهزة المتنوعة للعملاء.

Q8.20. Acceptance testing is done by

(a) Developers · (b) Customers · (c) Testers · (d) All of the above

(b) Customers

Acceptance testing is conducted by the end user or customer. It is used when the software is developed for a specific customer, who runs a series of tests to validate all requirements; these may range from ad hoc tests to a well-planned systematic series.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slides 12, 178; question on slide 193.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

اختبار القبول ينفذه الزبون أو المستخدم النهائي عندما يُطوَّر النظام لزبون محدد، وقد يكون عشوائياً أو منظماً.

Q8.21. One fault may lead to

(a) One failure · (b) No failure · (c) Many failure · (d) All of the above

(d) All of the above

One fault in software may lead to: (a) no failure (if unexecuted or masked), (b) a single failure, or (c) multiple failures across different operational runs. The manifestation of a fault depends dynamically on input data combinations and execution environment state.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.1 (p. 369); slide 9; question on slide 193.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

العيب البرمجي الواحد (Fault) قد لا يسبب أي فشل إن لم يُنفذ المسار أو تم تمويه النتيجة، أو قد يسبب فشلاً واحداً، أو يسبب انهيارات متعددة متكررة في بيئات مختلفة.

Q8.22. Test suite is

(a) Set of test cases · (b) Set of inputs · (c) Set of outputs · (d) None of the above

(a) Set of test cases

A test suite is a collection or set of test cases designed to test a program or module. It bundles inputs, execution conditions, and expected outcomes to verify compliance with a defined scope of requirements.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.2 (p. 369); slide 10; question on slide 193.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

حزمة الاختبار (Test Suite) هي مجموعة منظمة من حالات الاختبار (Set of test cases) تُجمع وتُنفذ سوياً للتحقق من وحدة أو نظام كامل.

Q8.23. Behavioral specification are required for:

(a) Modeling · (b) Verification · (c) Validation · (d) None of the above

(b) Verification

Publisher key: behavioral specifications are required for verification. The deck gives no sentence explaining this item, so it is a keyed fact; a behavioral specification is the reference the product's observable behavior is checked against.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; no explanatory slide; publisher key; question on slide 193.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

المفتاح الرسمي هو (b) Verification. السلايدات لا تشرح السبب بجملة صريحة، لذا لا تجادل المفتاح في الامتحان؛ تذكّر أن الخيار الذي يطابق حرفه هو Verification وليس Validation.

Q8.24. During the development phase, the following testing approach is not adopted

(a) Unit testing · (b) Bottom up testing · (c) Integration testing · (d) Acceptance testing

(d) Acceptance testing

Acceptance testing is not adopted during the development phase. Unit, bottom-up and integration testing are development-time approaches; acceptance testing involves the customer and is done after unit and integration testing.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370) and §8.5 (p. 435); slides 12, 170, 178; question on slide 194.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

أثناء التطوير تُستخدم Unit وBottom-up وIntegration. اختبار القبول (Acceptance) يخص الزبون ويأتي بعدها، لذلك هو الخيار غير المعتمد (d).

Q8.25. Which is not a functional testing technique?

(a) Boundary value analysis · (b) Decision table · (c) Regression testing · (d) None of the above

(c) Regression testing

Regression testing is not a functional testing technique. Boundary value analysis and decision tables are functional (black-box) techniques; regression testing is re-testing after changes.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3 (p. 371); slide 13; question on slide 194.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

BVA وDecision table تقنيات وظيفية، أما Regression فهو إعادة اختبار بعد التعديل وليس تقنية لاشتقاق الحالات من المواصفات.

Q8.26. Decision table are useful for describing situations in which:

(a) An action is taken under varying sets of conditions. · (b) Number of combinations of actions are taken under varying sets of conditions · (c) No action is taken under varying sets of conditions · (d) None of the above

(b) Number of combinations of actions are taken under varying sets of conditions

Decision tables are useful where a number of combinations of actions are taken under varying sets of conditions. They list conditions and actions as rules, so each combination of conditions is tied to its actions.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.3 (p. 395); slide 77; question on slide 194.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

جداول القرار تناسب الحالات التي تتغير فيها مجموعة الشروط وتُنفَّذ تركيبات مختلفة من الإجراءات. الفخ: الخيار (a) يتحدث عن إجراء واحد فقط.

Q8.27. One weakness of boundary value analysis and equivalence partitioning is

(a) They are not effective · (b) They do not explore combinations of input circumstances · (c) They explore combinations of input circumstances · (d) None of the above

(b) They do not explore combinations of input circumstances

One major weakness of boundary value analysis and equivalence class partitioning is that they do not explore combinations of input conditions. Both methods test variables largely in isolation under the single-fault assumption, missing defects caused by variable interactions.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.4 (p. 402); slide 92; question on slide 194.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

نقطة الضعف الجوهرية في تحليل القيم الحدية وتجزئة التكافؤ هي إهمال التفاعل بين الشروط المركبة، حيث تفترض كل طريقة فحص المتغير بمعزل عن بقية المتغيرات.

Q8.28. In cause effect graphing technique, cause & effect are related to

(a) Input and output · (b) Output and input · (c) Destination and source · (d) None of the above

(a) Input and output

In cause-effect graphing, cause and effect represent input condition and output action (or state change) respectively. A cause is a distinct input condition or equivalence class; an effect is an observable output or internal transformation.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3.4 (p. 402); slide 92; question on slide 195.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

في تقنية مخطط السبب والنتيجة (Cause-Effect Graphing)، يمثل «السبب» شرط الإدخال (Input)، ويمثل «الأثر» الإجراء أو المخرج الناتج (Output).

Q8.29. DD path graph is called as

(a) Design to Design Path graph · (b) Defect to Defect Path graph · (c) Destination to Destination Path graph · (d) Decision to decision Path graph

(d) Decision to decision Path graph

A DD path graph is a decision-to-decision path graph. It reduces the flow graph by combining runs of sequential nodes into single nodes, keeping the decision nodes.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.1 (p. 407); slides 112, 116; question on slide 195.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

DD = Decision to Decision. الرسم يدمج العقد المتتابعة ويُبقي عقد القرار. الفخ: لا تفكّ الاختصار إلى Design أو Defect أو Destination.

Q8.30. An independent path is

(a) Any path through the DD path graph that introduce at least one new set of processing statements or new conditions · (b) Any path through the DD path graph that introduce at most one new set of processing statements or new conditions · (c) Any path through the DD path graph that introduce at one and only one new set of processing statements or new conditions · (d) None of the above

(a) Any path through the DD path graph that introduce at least one new set of processing statements or new conditions

An independent path is any path through the DD path graph that introduces at least one new set of processing statements or new conditions.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.1 (p. 407); slides 117–118; question on slide 195.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

المسار المستقل يضيف على الأقل مجموعة جديدة من تعليمات المعالجة أو شروطاً جديدة. الفخ: at most وone and only one خياران خاطئان.

Q8.31. Cyclomatic complexity is developed by

(a) B.W.Boehm · (b) T.J.McCabe · (c) B.W.Lettlewood · (d) Victor Basili

(b) T.J.McCabe

Cyclomatic complexity was developed by T. J. McCabe. The deck introduces it as McCabe's cyclomatic metric, V(G) = e - n + 2P.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.2 (p. 422); slide 132; question on slide 195.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

مقياس التعقيد الدوري من وضع مكيب (T. J. McCabe).

Q8.32. Cyclomatic complexity is denoted by

(a) V(G)=e-n+2P · (b) V(G)= π+1 · (c) V(G)=Number of regions of the graph · (d) All of the above

(d) All of the above

All three formulations are valid: (a) V(G) = e - n + 2P, (b) V(G) = π + 1, where π is the number of predicate nodes, and (c) V(G) = the number of regions of the flow graph.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.2 (p. 422); slides 132, 138; question on slide 196.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

الطرق الثلاث صحيحة: e-n+2P (حيث P عدد المكونات المتصلة)، وπ+1 (حيث π عدد عقد القرار)، وعدد المناطق. لذلك الإجابة (d).

Q8.33. The equation V(G)= π+1 of cyclomatic complexity is applicable only if every predicate node has

(a) two outgoing edges · (b) three or more outgoing edges · (c) no outgoing edges · (d) none of the above

(a) two outgoing edges

The equation V(G) = π + 1 applies only if every predicate node has two outgoing edges. A node with three or more outgoing edges breaks the simple +1 rule.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.2 (p. 422); slide 138; question on slide 196.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

صيغة π+1 تصح فقط إذا كان لكل عقدة قرار مخرجان اثنان بالضبط.

Q8.34. The size of the graph matrix is

(a) Number of edges in the flow graph · (b) Number of nodes in the flow graph · (c) Number of paths in the flow graph · (d) Number of independent paths in the flow graph

(b) Number of nodes in the flow graph

The size of a graph matrix is an $N \times N$ square matrix, where N is the number of nodes in the flow graph. Each row and column corresponds directly to a node.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.3 (p. 426); slide 147; question on slide 196.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

مصفوفة الرسم البياني (Graph Matrix) هي مصفوفة مربعة بأبعاد $N \times N$، حيث يمثل N عدد العقد الإجمالي في مخطط تدفق البرنامج.

Q8.35. Every node is represented by

(a) One row and one column in graph matrix · (b) Two rows and two columns in graph matrix · (c) One row and two columns in graph matrix · (d) None of the above

(a) One row and one column in graph matrix

In a graph matrix, every node is represented by exactly one row and one column. The entry at (i, j) indicates whether a directed edge connects node i to node j.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.3 (p. 426); slide 147; question on slide 197.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

في مصفوفة الرسم، تُمثل كل عقدة بصف واحد وعمود واحد حصراً؛ وتُسجل التوصيلات بين العقدة i والعقدة j في الخلية المتقاطعة.

Q8.36. Cyclomatic complexity is equal to

(a) Number of independent paths · (b) Number of paths · (c) Number of edges · (d) None of the above

(a) Number of independent paths

Cyclomatic complexity $V(G)$ is equal to the number of linearly independent paths through the program flow graph. It establishes the upper bound for the number of test cases required to achieve complete branch coverage.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.2 (p. 422); slide 134; question on slide 197.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

التعقيد الدوري $V(G)$ يساوي رياضياً عدد المسارات المستقلة خطياً (Number of independent paths) في فضاء المسارات الأساسي للبرنامج.

Q8.37. Data flow testing is related to

(a) Data flow diagrams · (b) E-R diagrams · (c) Data dictionaries · (d) none of the above

(d) none of the above

Data flow testing is not related to data flow diagrams, E-R diagrams, or data dictionaries. Data flow testing is a structural (white-box) technique focusing on the lifecycle of program variables (definitions, uses, and kills) along paths in control flow graphs.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.4 (p. 430); slide 155; question on slide 197.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

فحص تدفق البيانات (Data Flow Testing) هو فحص صندوق أبيض لمسارات المتغيرات (DEF/USE) في كود البرنامج، ولا علاقة له بمخططات تدفق البيانات (DFD) أو مخططات الكيانات (ERD).

Q8.38. In data flow testing, objective is to find

(a) All dc-paths that are not du-paths · (b) All du-paths · (c) All du-paths that are not dc-paths · (d) All dc-paths

(c) All du-paths that are not dc-paths

In data flow testing, a key objective is to uncover data anomalies, which requires examining all du-paths that are not definition-clear (dc) paths. If a variable is re-defined along a path before being used, an anomaly or redundant definition exists.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.4 (p. 430); slides 157–158; question on slide 197.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

الهدف في فحص تدفق البيانات هو كشف التشوهات (Data anomalies) باختبار المسارات التي يعاد فيها تعريف المتغير دون استخدامه (المسارات التي ليست dc-paths نظيفة).

Q8.39. Mutation testing is related to

(a) Fault seeding · (b) Functional testing · (c) Fault checking · (d) None of the above

(a) Fault seeding

Mutation testing is closely related to fault seeding. In mutation testing, artificial syntactic faults (mutants) are deliberately inserted into programs to assess the adequacy and effectiveness of a test suite in 'killing' the mutated programs.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.4.5 (p. 430); slide 167; question on slide 198.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

فحص التحوير (Mutation testing) يعتمد على غرس الأخطاء البرمجية عمداً (Fault seeding) لتقييم كفاءة مجموعة الاختبار في اكتشاف وقتل النسخ المشوهة (Mutants).

Q8.40. The overhead code required to be written for unit testing is called

(a) Drivers · (b) Stubs · (c) Scaffolding · (d) None of the above

(c) Scaffolding

The overhead code required to be written for unit testing (consisting of drivers and stubs) is called scaffolding. Drivers simulate calling modules, while stubs simulate subordinate called modules.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.5.1 (p. 436); slide 172; question on slide 198.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

الكود الإضافي المساند المكتوب لتنفيذ فحص الوحدة (Unit testing)، والذي يتضمن برامج القيادة (Drivers) والمحاكيات البديلة (Stubs)، يُسمى السقالات البرمجية (Scaffolding).

Q8.41. Which is not a debugging techniques

(a) Core dumps · (b) Traces · (c) Print statements · (d) Regression testing

(d) Regression testing

Regression testing is a testing activity, not a debugging technique. Core dumps, traces and print statements are debugging aids; debugging begins after a failure symptom is found and locates the place and cause of the error.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.7 (p. 441); slides 180–185; question on slide 198.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

Regression فحص وليس أسلوب تنقيح. التنقيح يبدأ بعد ظهور العَرَض ويحدد مكان الخطأ وسببه.

Q8.42. A break in the working of a system is called

(a) Defect · (b) Failure · (c) Fault · (d) Error

(b) Failure

A break or observable departure in the normal working/operation of a system is called a failure. A fault is the internal static defect in code; an error is a human misaction; a failure is the dynamic breakdown observed during execution.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.1 (p. 369); slide 9; question on slide 198.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

التوقف أو العطل الظاهر في عمل المنظومة أثناء التشغيل يُسمى فشلاً (Failure). الفشل هو العرض الخارجي الملاحظ، والخلل (Fault) هو السبب الداخلي في الكود.

Q8.43. Alpha and Beta testing techniques are related to

(a) System testing · (b) Unit testing · (c) acceptance testing · (d) Integration testing

(c) acceptance testing

Alpha and Beta testing techniques are specialized forms of acceptance testing. They assess software readiness from user perspectives before formal commercial distribution.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slides 12, 178; question on slide 198.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

اختبارات ألفا وبيتا هي أشكال وتطبيقات مباشرة تابعة لاختبار القبول (Acceptance testing) لتقييم رضا المستخدمين عن المنظومة.

Q8.44. Which one is not the verification activity

(a) Reviews · (b) Path testing · (c) Walkthrough · (d) Acceptance testing

(d) Acceptance testing

Acceptance testing is not a verification activity; it is a dynamic validation activity. Reviews, walkthroughs, inspections, and path tracing are verification activities examining artifacts for specification conformance.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.3 (p. 370); slides 11, 178; question on slide 199.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

اختبار القبول (Acceptance testing) هو نشاط فحص تشغيلي تابع للـ Validation وليس نشاط تدقيق مكتبي تابع للـ Verification مثل المراجعات (Reviews) والتدقيق (Walkthrough).

Q8.45. Testing the software is basically

(a) Verification · (b) Validation · (c) Verification and validation · (d) None of the above

(c) Verification and validation

Testing software is basically an integrated discipline encompassing both verification and validation (V&V). Verification confirms structural compliance with design specifications, while validation confirms behavioral satisfaction of customer requirements.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.3 (p. 370); slide 11; question on slide 199.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

فحص البرمجيات يشمل كلاً من التحقق الهيكلي والتحقق الوظيفي (Verification and Validation) معاً لضمان الجودة الشاملة للمنتج.

Q8.46. Integration testing techniques are

(a) Topdown · (b) Bottom up · (c) Sandwich · (d) All of the above

(d) All of the above

Integration testing techniques include: (a) top-down integration, (b) bottom-up integration, and (c) sandwich (hybrid) integration. All of the above are standard structural strategies for combining and testing interacting modules.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.5.2 (p. 437); slide 175; question on slide 199.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

استراتيجيات فحص التكامل تشمل الفحص التنازلي (Top-down)، الفحص التصاعدي (Bottom-up)، والفحص الهجين/الشطيرة (Sandwich).

Q8.47. Functionality of a software is tested by

(a) White box testing · (b) Black box testing · (c) Regression testing · (d) None of the above

(b) Black box testing

The functionality of software is tested by black-box (functional) testing. Black-box testing evaluates system responses strictly against functional requirements without requiring knowledge of internal code constructs.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.3 (p. 371); slide 13; question on slide 199.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

فحص وظائف النظام وسلوكه الخارجي يتم حصراً عبر فحص الصندوق الأسود (Black box testing) بالاعتماد على متطلبات النظام.

Q8.48. Top down approach is used for

(a) Development · (b) Identification of faults · (c) Validation · (d) Functional testing

(b) Identification of faults

Publisher key: (b) identification of faults. The deck shows top-down as one of the three integration approaches (Fig. 30) but gives no sentence tying it to this option, so treat it as a keyed fact.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.5.2 (p. 437); slide 175; question on slide 199.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

مفتاح الناشر (b). السلايد 175 يعرض الفحص التنازلي (Top-down) كأحد أساليب التكامل دون جملة تفسّر هذا الخيار، فاحفظ المفتاح.

Q8.49. Thread testing is used for testing

(a) Real time systems · (b) Object oriented systems · (c) Event driven systems · (d) All of the above

(d) All of the above

Publisher key: (d) all of the above. The chapter text does not define thread testing; the term appears only in this question, so treat it as a keyed fact.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; no explanatory slide; publisher key; question on slide 200.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

المفتاح الرسمي (d). مصطلح Thread testing يرد في هذا السؤال فقط ولا يُشرح في نص الفصل، فاحفظه كمفتاح.

Q8.50. Testing of software with actual data and in the actual environment is called

(a) Alpha testing · (b) Beta testing · (c) Regression testing · (d) None of the above

(b) Beta testing

Testing of software with actual data and in the actual live operational environment is called beta testing. It exposes the product to realistic operational stress in customer environments without developer intervention.

Source: Aggarwal & Singh (3rd ed.), question printed pp. 451–455, publisher key p. 499; §8.2.4 (p. 370); slide 12; question on slide 200.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

فحص البرمجيات بالبيانات الفعلية الحقيقية وفي بيئة العمل الواقعية بواسطة المستخدمين يُسمى اختبار بيتا (Beta testing).
